

**PERSONALIZED ELECTRIC VEHICLE ENERGY CONSUMPTION ESTIMATION  
FRAMEWORK THAT INTEGRATES DRIVER BEHAVIOR WITH MAP DATA**

by

Sreechakra Vasudeva Raju Rachavelpula

Applied Project report submitted to the Faculty of the Graduate School of the



in fulfillment of the requirements for the degree of  
Master of Science in Computer and Information Sciences

Supervised by: Sangwhan Cha, Ph.D.

2026

# Abstract

This project aims to develop a personalized energy-consumption estimation framework for Battery Electric Vehicles (BEVs) by modeling driver behavior using machine-learning techniques and integrating it with map-based contextual data for a selected route. By analyzing a driver’s historical velocity patterns, the proposed model predicts future vehicle speeds, enabling improved understanding of how individuals navigate road features such as intersections, curves, and grade changes. When combined with detailed route geometry and regulatory information, this approach yields highly accurate predictions of vehicle energy consumption and State-of-Charge (SOC) evolution.

To achieve this, the thesis introduces an end-to-end pipeline that integrates route selection, map-feature extraction and processing, a rule-based reference velocity generator, a PID controller-like vehicle dynamics simulator, a Bidirectional Long Short-Term Memory (BiLSTM) driver-behavior model, and a physics-based backward energy consumption model. The system leverages OpenStreetMap and Valhalla APIs to obtain high-resolution road attributes—including curvature, slope, speed limits, and traffic-control indicators—which are fused then with past driving behavior and future road context to generate driver-specific velocity trajectories. These trajectories are then used to compute tractive power, regenerative braking, energy consumption, and SOC depletion along the route.

The framework is evaluated across three representative driving environments: urban, freeway, and hilly terrain. Results show that the BiLSTM model captures key driver behavioral characteristics such as deceleration at intersections, speed-limit tracking, and road grade dependent control responses. The energy model produces physically consistent power and SOC profiles, including regeneration during downhill segments and increased power demand during climbs. Together, these results demonstrate that combining learned driver behavior with map-aware, physics-based modeling significantly improves the accuracy and personalization of BEV range estimation. Overall, this work establishes a modular and extensible foundation for personalized BEV energy forecasting.

# Contents

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                       | <b>ii</b> |
| <b>Table of Contents</b>                                              | <b>iv</b> |
| <b>1 Introduction</b>                                                 | <b>1</b>  |
| 1.1 Introduction . . . . .                                            | 1         |
| 1.2 Background and Motivation . . . . .                               | 2         |
| 1.3 Problem Statement . . . . .                                       | 3         |
| 1.4 Research Objective: . . . . .                                     | 3         |
| 1.5 Thesis Contributions . . . . .                                    | 4         |
| <b>2 Literature Review</b>                                            | <b>5</b>  |
| 2.1 Parametric and Classical Approaches to Speed Prediction . . . . . | 5         |
| 2.2 Machine Learning for Speed Prediction . . . . .                   | 6         |
| 2.3 Incorporating Map-Based Features . . . . .                        | 7         |
| 2.4 Driver-Specific Behavioral Modeling . . . . .                     | 7         |
| 2.5 Bidirectional LSTM Architectures . . . . .                        | 8         |
| 2.6 Energy Consumption and Range Estimation Models . . . . .          | 9         |
| 2.7 Summary . . . . .                                                 | 9         |
| <b>3 System Specifications</b>                                        | <b>10</b> |
| 3.1 System Overview . . . . .                                         | 10        |
| 3.2 Functional Requirements and Development Approach . . . . .        | 10        |
| 3.3 Non-Functional Requirements and Development Strategies . . . . .  | 15        |
| 3.4 System Modules and Interactions . . . . .                         | 18        |
| <b>4 System Design and Implementation</b>                             | <b>23</b> |
| 4.1 Map Display Handler . . . . .                                     | 23        |
| 4.2 Route Processor . . . . .                                         | 24        |
| 4.3 Velocity Profile Generator . . . . .                              | 30        |
| 4.4 PID Controller . . . . .                                          | 31        |
| 4.4.1 Command Velocity Generator . . . . .                            | 32        |
| 4.4.2 Vehicle Dynamics Model . . . . .                                | 36        |
| 4.5 Driver LSTM Module . . . . .                                      | 38        |
| 4.5.1 Long Short-Term Memory Networks . . . . .                       | 38        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 4.5.2    | Bidirectional LSTM Networks . . . . .                                 | 40        |
| 4.5.3    | Prediction Framework . . . . .                                        | 40        |
| 4.5.4    | Proposed Architecture . . . . .                                       | 41        |
| 4.5.5    | Implemented Architecture . . . . .                                    | 43        |
| 4.5.6    | Training the Driver LSTM Model . . . . .                              | 44        |
| 4.5.7    | Inference Procedure . . . . .                                         | 45        |
| 4.6      | BEV Energy Consumption Model . . . . .                                | 46        |
| 4.6.1    | Tractive Power . . . . .                                              | 47        |
| 4.6.2    | Electric Motor Power . . . . .                                        | 47        |
| 4.6.3    | Battery Power and Auxiliary Loads . . . . .                           | 48        |
| 4.6.4    | Energy Consumption Estimation . . . . .                               | 48        |
| 4.6.5    | SOC Dynamics . . . . .                                                | 49        |
| 4.6.6    | Integration with Driver-Specific Velocity Predictions . . . . .       | 49        |
| <b>5</b> | <b>Testing and Results</b>                                            | <b>50</b> |
| 5.1      | Development of Unit Test Cases . . . . .                              | 50        |
| 5.1.1    | Map Display Handler Test cases . . . . .                              | 50        |
| 5.1.2    | Route Processor Test Cases . . . . .                                  | 51        |
| 5.1.3    | Velocity Profile Generator Test Cases . . . . .                       | 51        |
| 5.1.4    | PID Controller Test Cases . . . . .                                   | 52        |
| 5.1.5    | Driver LSTM Module Test Cases . . . . .                               | 52        |
| 5.1.6    | BEV Energy Consumption and SOC Estimation Module Test Cases . . . . . | 53        |
| 5.2      | Results . . . . .                                                     | 53        |
| 5.2.1    | Urban Route . . . . .                                                 | 55        |
| 5.2.2    | Freeway Route . . . . .                                               | 59        |
| 5.2.3    | Hilly Terrain Route . . . . .                                         | 60        |
| <b>6</b> | <b>Conclusions and Future work</b>                                    | <b>65</b> |
|          | <b>Bibliography</b>                                                   | <b>67</b> |

# Chapter 1

## Introduction

### 1.1 Introduction

The transition from internal combustion engine (ICE) vehicles to battery electric vehicles (BEVs) represents one of the most significant technological shifts in modern transportation. BEVs offer the potential to reduce greenhouse gas emissions, improve energy efficiency, and decrease dependence on fossil fuels. However, their widespread adoption continues to be limited by users' concerns regarding driving range and charging availability, a phenomenon commonly referred to as *range anxiety* [18, 21]. Accurate prediction of vehicle range and energy consumption is therefore a critical enabler for confident BEV adoption, efficient trip planning, and optimized charging-infrastructure deployment [25].

Existing BEV range estimation algorithms typically rely on recent consumption statistics or average driving conditions rather than contextual factors such as route geometry, traffic flow, or individual driver behavior. As a result, the estimated range often diverges substantially from the actual achievable range, particularly in urban environments characterized by stop-and-go movement, variable gradients, and heterogeneous driving styles [16, 3]. Over or under-estimation of the remaining state of charge (SOC) not only causes discomfort and planning inefficiencies but also accelerates driver disengagement from electric mobility [21].

To address these shortcomings, researchers have increasingly focused on developing *context-aware* and *data-driven* range estimation frameworks. These frameworks account for geographic, environmental, and behavioral factors that influence vehicle energy dynamics. Among them, the inclusion of map-based data such as road curvature, slope, speed limits, and intersection density has shown significant improvements in energy prediction accuracy [35, 28, 25]. Furthermore, driver behavior has been recognized as one of the most decisive variables in determining instantaneous and trip-level energy consumption [27, 34, 29].

A key insight from the literature is that aggressive acceleration, harsh braking, and inconsistent speed maintenance can reduce BEV range by 20–40% compared to cautious driving [1, 34]. Consequently, any attempt to achieve realistic range estimation must integrate both environmental and behavioral dimensions of vehicle operation. This observation forms the basis of this research: the development of a personalized driver-behavior-informed energy consumption estimation algorithm that incorporates map-based contextual data and data-driven behavioral modeling.

## 1.2 Background and Motivation

Modern electric vehicles are equipped with advanced sensor suites capable of logging detailed telemetry, including velocity, acceleration, throttle, brake pressure, and GPS data. The proliferation of publicly available map databases such as OpenStreetMap (OSM) and tools such as the Valhalla routing engine enables the extraction of route-specific features like elevation profiles, curvature, and speed limits. These developments create an opportunity to link driver behavior, road context, and vehicle dynamics through a unified predictive framework.

Prior research has demonstrated that combining historical driving data with energy models significantly improves estimation accuracy [2, 37]. However, most existing models remain either (a) generalized across multiple drivers—with limited personalization—or (b) dependent on deterministic assumptions about future driving speed. Few studies have attempted to construct driver-specific predictive models that use both *past driving behavior* and *future road geometry* to anticipate velocity changes and subsequent energy use.

Machine learning techniques, particularly recurrent neural networks (RNNs) and their Long Short-Term Memory (LSTM) variants, are well suited for such sequence modeling tasks [23, 40]. LSTM architectures can retain long-term temporal dependencies and model complex nonlinear relationships between driving patterns, vehicle response, and external context. When combined with route information, this framework can forecast future velocity profiles under specific driving styles, leading to accurate energy consumption and SOC estimation for an individual driving style. [34, 29].

In a previous approach to this problem (implemented in GRAD 695), we successfully built the foundational pipeline for such a model by developing:

- A **Map data extraction tool** for retrieving route geometry using OpenStreetMap and the Valhalla routing engine.
- A **Velocity profile generator** approximating typical speed behavior given road conditions.
- A **PID-based vehicle dynamics model** that simulates realistic acceleration, deceleration, and jerk while adhering to physical constraints.

Here, we extend this framework into a machine-learning based architecture that uses driver-specific data to replace the approximated velocity generator with an LSTM-based predictor in order to achieve truly personalized BEV range estimation.

### 1.3 Problem Statement

Despite considerable progress in BEV range estimation, current approaches exhibit several critical gaps:

1. **Lack of personalization:** Most commercial systems estimate range using averaged consumption rates, failing to account for individual driver behavior. This leads to large systematic errors for aggressive or conservative drivers [27, 34].
2. **Limited incorporation of map-based features:** Many predictive models ignore the influence of upcoming road conditions such as gradient, curvature, and traffic control elements such as stop signs, traffic signals etc., which significantly affect speed and energy consumption [35, 28].
3. **Short prediction horizons:** Traditional models often operate on short-term time frames (5–30s) and cannot provide accurate long-term forecasts necessary for trip-level range estimation [14, 30].
4. **Compounding estimation errors:** Deterministic or Markov models propagate uncertainty over time, leading to accumulating deviations from actual vehicle behavior during long trips [3].

These limitations motivate the need for a novel framework that integrates future road context and personalized driver models into a single predictive pipeline.

### 1.4 Research Objective:

The primary objective of this research is to develop a machine-learning based personalized energy consumption estimation framework for electric vehicles that integrates:

1. Driver-specific behavioral learning using past velocity sequences and acceleration patterns.
2. Map-based contextual information, including both past and future road geometry extracted from OpenStreetMap and Valhalla APIs.
3. A physics based energy consumption model that uses predicted speed and acceleration trajectories to estimate SOC depletion and energy consumption.

The proposed architecture in Chapter 3 seeks to answer these questions by coupling data-driven predictive modeling with physically interpretable energy estimation. The end goal is a robust, personalized, and context-aware BEV energy consumption estimation system capable of adapting to different drivers, terrains, and routes.

## 1.5 Thesis Contributions

The contributions of this research can be summarized as follows:

1. Integration of map-based contextual attributes, including slope, curvature, and speed limits, derived from OpenStreetMap and the Valhalla routing framework.
2. Development of a deep-learning based velocity prediction framework utilizing Bidirectional Long Short-Term Memory (LSTM) networks to model past and future dependencies of the driving behavior on map-based contextual features.
3. Formulation of an extended backward energy model for the estimation of power demand, energy usage, and state-of-charge under personalized speed trajectories.

Overall, this work aims to bridge the gap between data-driven driver behavior modeling, map-aware prediction, and physics based energy consumption estimation—moving toward a truly personalized BEV range estimation system.



## Chapter 2

# Literature Review

Predicting vehicle speed and estimating energy consumption have become central research problems in intelligent transportation systems, battery electric vehicle (BEV) energy management, and hybrid electric vehicle (HEV) control. Over the past two decades, the literature has evolved from classical parametric models toward data-driven, machine-learning based approaches that leverage large-scale driving datasets, map information, and driver-specific behavioral patterns. This chapter reviews the major bodies of work relevant to the present thesis: parametric and classical approaches to speed prediction, machine-learning based speed prediction, incorporation of map-based features, driver-specific behavioral modeling, bidirectional LSTMs and deep-learning architectures for long-term prediction, and energy consumption and range estimation models. Together, these strands of research form the foundation for a personalized, map-aware, long-horizon vehicle speed prediction and energy consumption estimation framework.

### 2.1 Parametric and Classical Approaches to Speed Prediction

Early research on vehicle speed prediction relied heavily on physics-based and rule-based models, which attempted to capture vehicle longitudinal dynamics using simplified representations of traction, drag, rolling resistance, and grade. Classical models such as the Intelligent Driver Model (IDM) [19] and its variants provided analytical formulations for car-following behavior, acceleration limits, and safe headway maintenance. These models were widely used in microscopic traffic simulation and eco-driving studies due to their interpretability and low computational cost.

However, these methods often proved inadequate for modeling the stochasticity of real-world driving. As Deufel et al. [5] noted, parametric models assume idealized conditions and fail to adapt to diverse environments – especially urban scenarios characterized by stop-and-go traffic, unpredictable pedestrian crossings, and signalized intersections. Similarly, Bhuiyan and Kabir [17] observed that classical rule-based models cannot capture external environmental effects such as road curvature, surface quality, or weather, all of which significantly alter vehicle speed.

Markov-chain and stochastic frameworks later bridged the gap between deterministic and data-driven approaches. Studies such as Deufel et al. [4] and Wu et al. [35] demonstrated that discrete-state Markov models can reproduce realistic short-term velocity or gradient transitions, yielding plausible synthetic drive cycles. Yet, the inherent assumption of first-order dependence causes accuracy to degrade at long horizons, as key behavioral or environmental dependencies are unmodeled.

Despite these limitations, classical and stochastic approaches established the foundation for later hybrid and data-driven models by codifying fundamental relationships between vehicle states, driver inputs, and roadway geometry [18]. They remain useful baselines for examining energy consumption sensitivity to speed and gradient variations.

## 2.2 Machine Learning for Speed Prediction

With the availability of high-resolution telemetry data from connected vehicles, machine learning (ML) has emerged as the dominant paradigm for speed prediction. Early models – including Feedforward Neural Networks (FNNs) and Support Vector Machines (SVMs) – were applied to derive nonlinear mappings between past driving parameters (speed, acceleration, throttle input) and near-term velocities. While effective in limited contexts, these models assume temporal independence and are unable to capture sequence continuity or long-term behavioral trends.

Building upon these, recurrent neural networks, especially Long Short-Term Memory (LSTM) architectures, have become the standard for sequential prediction tasks. LSTMs can store and selectively recall relevant past information using gated states, overcoming the vanishing-gradient limitation of vanilla RNNs. Studies such as Li et al. [38] and Deufel et al. [5] have shown that LSTM networks substantially outperform shallow neural networks and SVMs for short to mid-term speed forecasting, and are well-suited for driver-specific modeling.

Further extensions—such as the LSTM encoder-decoder and stacked LSTM frameworks allow modeling contextual dependencies between road geometry and vehicle kinematics. For instance, Paparusso et al. [24] fused temporal vehicle dynamics with future road curvature using 3-D spline road representations, producing smoother long-horizon predictions. Similarly, Vransky et al. [30] employed Bidirectional LSTMs (BiLSTMs) to synthesize route-specific velocity sequences that generalize across unseen road conditions.

Recent comparative analyses [14] confirmed that LSTM-based methods achieve a favorable trade-off between accuracy and computational efficiency – particularly for applications such as pre-

dictive eco-driving and energy-aware trajectory planning. Nevertheless, long-horizon predictions beyond 30s remain challenging due to cumulative prediction drift and the absence of look-ahead map context.

## 2.3 Incorporating Map-Based Features

A major limitation of early LSTM models lies in their reliance on past vehicle dynamics alone. Yet, long-term velocity evolution fundamentally depends on upcoming road geometry, including curvature, slope, intersection density, speed limits, and traffic signal locations. Map-based features therefore provide indispensable context for anticipating how a driver will adjust speed in response to the road ahead.

Several studies have highlighted this need. Deufel et al. [5] integrated OpenStreetMap (OSM) and Shuttle Radar Topography Mission (SRTM) data for slope, curvature, and elevation extraction, demonstrating substantial improvements in predictive accuracy. Wu et al. [36] showed that incorporating terrain grade and curvature provides more robust energy-consumption estimates by accounting for the gravitational and frictional forces associated with road geometry. Similarly, Paparusso et al. [24] established that future road features (e.g., turns, gradient changes) are essential for context-aware long-term forecasts.

Further confirming this, Shen et al. [26] used transformer-based frameworks with embedded road feature tokens (traffic lights, stop/yield signs) for improved acceleration modeling, while Hou et al. [15] incorporated map-based trajectory augmentation to represent macro-level mobility patterns. Empirical studies, such as those of Polat et al. [25], affirm that factors such as topography, surface type, and traffic congestion strongly influence vehicle energy efficiency and range estimation. Consequently, incorporating both past dynamics and future road geometry—an approach also adopted in this thesis—has become the state-of-the-art strategy for accurate long-horizon speed and energy prediction models.

## 2.4 Driver-Specific Behavioral Modeling

Individual driving styles play a significant role in determining vehicle energy consumption and range. Aggressive drivers, characterized by high acceleration and jerk, have been observed to reduce electric vehicle range by up to 20–40%, whereas conservative drivers extend it through smoother control inputs [27, 29, 34].

Lv et al. [20] introduced a cyber-physical systems framework linking driving style classification

to energy management, while Wang and Lukic[33] identified key behavioral indicators such as pedal aggressiveness and speed variance. They demonstrated behavior-induced variations in energy use is comparable to those caused by environmental conditions. Extending these findings, Guo et al. [13] proposed an adaptive BEV energy consumption model integrating driver behavior with road geometry, yielding less than 5% mean prediction error.

From a data-science perspective, driver-specific modeling evolved from static clustering (aggressive/moderate/calm) toward temporal sequence modeling using recurrent networks. Li et al. [38] and Paparusso et al.[24] showed that incorporating driver-specific history substantially improves speed prediction robustness, while Shen et al. [26] found that embedding past acceleration distributions enhances energy-prediction accuracy.

Yet, despite these advances, few works explicitly encode driver identity as a learnable feature, instead aggregating data across multiple drivers, effectively diluting driver-specific distinctions. This gap motivates the personalized speed-prediction objective of the present thesis, where past driver behavior and map-derived road features jointly inform an individualized velocity model.

## 2.5 Bidirectional LSTM Architectures

Bidirectional LSTMs (BiLSTMs) process sequences in both forward (past  $\rightarrow$  future) and backward (future  $\rightarrow$  past) directions, allowing the model to use future context when inferring current behavior. For vehicle-speed prediction, this mirrors how drivers mentally anticipate the upcoming road geometry when modulating speed.

Vrany et al. [30] demonstrated that BiLSTMs yield smoother, more realistic velocity profiles than unidirectional networks. Heckelmann et al. [14] compared multiple encoder-decoder structures and showed that incorporating bidirectional layers improves accuracy on complex urban segments while requiring fewer parameters than transformer-based counterparts. Building on the same principle, Paparusso et al. [24] leveraged dual encoders for past dynamics and future road geometry, effectively aligning with the architecture proposed in this thesis.

While transformer architectures (e.g., Shen et.al.,[26]) provide powerful sequence-to-sequence learning across long contexts, their heavy parameterization and data requirements make BiLSTMs a pragmatic alternative for embedded or real-time mobility applications.

## 2.6 Energy Consumption and Range Estimation Models

Accurate energy consumption and state-of-charge (SOC) estimation is vital for addressing range anxiety and improving user confidence [21, 18]. Although BEVs deliver high drivetrain efficiency, their limited battery capacity means that uncertainty in speed or load prediction directly impacts range accuracy.

Physics-based backward vehicle models, such as Wang et al. [32], decompose power demand into aerodynamic, inertial, and gravitational components driven by speed, acceleration, and grade. Wu et al. [36] emphasized the role of curvature and grade on vehicle traction loss and energy usage. Similarly, Al-Wreikat et al. [34] reported strong real-world correlations between acceleration aggressiveness, average speed, ambient temperature, and SOC reduction.

Complementing these, Fiori et al. [11] presented a comprehensive power-based EV Energy consumption model (VT-CPEM) that maps instantaneous speed, acceleration, and slope into power demand without requiring component-level efficiency maps. Such quasi-steady approaches, further validated by Valladolid et al. [29], offer computational tractability for route-level range estimation.

Beyond deterministic models, modern methods now focus on predictive energy management, in which future velocity and power demand are jointly forecasted. Zhai et al. [39] introduced a predictive energy management algorithm for hybrid energy storage systems using pattern-based velocity forecasts, highlighting that accurate speed prediction directly improves energy allocation and range forecasting. Similarly, Tannahill et al. [27] developed a dynamic BEV range estimation framework that integrates route conditions and driver influence, validating the importance of behavioral and contextual modeling.

## 2.7 Summary

Across the existing literature, there is a clear evolution from rule-based and physically interpretable models toward machine-learning-enabled, driver behavior-aware, and map-informed frameworks. Incorporating driver-specific behavioral features, LSTM-based temporal modeling, and road context now constitutes the state of the art in BEV energy prediction. However behavioral variability and route uncertainty remain major sources of range estimation error. Addressing these limitations through a personalized BiLSTM framework that jointly encodes past driver behavior and future road geometry forms the central research contribution of this thesis.

## Chapter 3

# System Specifications

This chapter presents the comprehensive system specifications for the personalized Battery Electric Vehicle (BEV) energy consumption estimation framework that integrates driver behavior modeling with map-based contextual data. It provides both the functional and non-functional requirements essential for implementing the system, linking each to the defined technical stack and underlying software architecture. Furthermore, it details how these requirements will be realized, maintained, and validated within the proposed modular design.

### 3.1 System Overview

The proposed system estimates personalized BEV range by combining three core components:

- **Map-Based Route Data:** Collected from OpenStreetMap (OSM) and processed through the Valhalla Routing Engine to capture route geometry and road characteristics.
- **Driver Behavior Model:** Utilizes an LSTM recurrent neural network to capture individual driving styles based on historical velocity data and map features.
- **BEV Energy Model:** Employs a physics-based formulation to simulate tractive power, regenerative braking, and SOC depletion for the predicted speed profile.

### 3.2 Functional Requirements and Development Approach

The following functional requirements (FRs) guide how key system functions will be implemented using the defined technical stack. For each FR, the practical development method and associated tools are described in detail.

## FR-001: Route Selection

**Requirement:** The system shall provide an intuitive route selection interface using OpenStreetMap (OSM) for users to specify a start and destination by typing in the address or selecting manually by dropping a pointer on the map. The system shall then extract all the GPS coordinates for the calculated route and store them locally in JSON format.

**Rationale:** The route selection is the most important input provided by user to the application. Therefore route selection should be very intuitive and hassle-free for the user.

### Implementation Strategy:

- Implement OSM access through Python Selenium and ChromeDriver to automatically capture route coordinates from the interactive map.
- The collected GPS coordinates are exported as a `.geojson` file for downstream modules.
- Use event listeners to detect user route selection and user timeout.

## FR-002: Map Feature Extraction

**Requirement:** The system shall retrieve node and edge features for the selected route from a hosted Valhalla Routing Engine instance. The Valhalla Routing engine instance can be locally hosted, but in the event of failure the open-source Valhalla server hosted at <https://valhalla1.openstreetmap.de> shall be used. Extracted features shall include but not be limited to legal speed limits, default edge speed, road classification, heading, elevation profiles, curvature, presence of traffic signals, stop signs, yield signs, roundabouts, road links and intersection complexity.

**Rationale:** Accurate and granular map data is essential for modeling driver behavior and estimating energy consumption. Route-specific features directly influence vehicle kinematics and energy usage.

### Implementation Strategy:

- Utilize `requests` and `json` Python libraries to query Valhalla's `Location`, `Map Matching (Trace)` and `Elevation` APIs.
- Extract edge geometry, curvature, elevation, and speed limits.
- Organize data into a Pandas `DataFrame` indexed by the distance along the route.

## FR-003: Feature Processing and Normalization

**Requirement:** The system shall process raw map features into normalized, model-ready tensors that can be used for training the LSTM based driver model and also for inference. This includes creation of sequences for batch training and inference, spatial alignment of features, normalization using training-time scalers, and appropriate encoding of categorical variables. The processor shall ensure schema consistency with the trained LSTM model during inference.

**Rationale:** Machine learning models require consistent and normalized input formats. This module ensures that inference-time data matches the structure and scaling of training data. This enables accurate velocity and energy consumption predictions for the BEV.

### Implementation Strategy:

- Use NumPy for array manipulation and Pandas for schema validation.
- Use Scikit-learn scalers to normalize continuous variables such as grade, curvature, heading, and speed limits.
- Implement one-hot encoding for categorical features such as road class.

## FR-004: Driver-specific Velocity Prediction

**Requirement:** The system shall implement a pretrained LSTM-based driver behavior model that predicts a driver-specific speed profile over the selected route. The model shall accept the relevant processed map features and the historical driving data over a pre-configured past distance span (e.g. previous 50-100 meters) and the future processed map features for a pre-configured horizon (e.g. next 50-100 meters) as input and output a multi-step speed forecast over the same horizon. The input past distance span of the training data and output prediction horizon shall also be the hyperparameters that need to be tuned for optimal prediction. For batch inferencing, a simulated vehicle speed profile shall be used as the past driving data instead of autoregressive inferencing to reduce latency.

**Rationale:** Driver behavior significantly affects energy consumption. Personalized speed profiles enable more accurate modeling of acceleration, deceleration, and cruising behavior, which are critical for energy estimation.

### Implementation Strategy:

- Retrieve the best model checkpoint weights from Google Cloud Storage and store them locally at the point of deployment for inferencing.



- Create input sequences of past data of shape {batch size, past steps, number of road and vehicle kinematics features} and future data of shape {batch size, future steps, number of road features} for both training and inference using Pandas, Scikit and Tensorflow utilities.

## FR-005: BEV Energy Consumption Computation and SOC Projection

**Requirement:** The system shall simulate BEV energy consumption using a physics-based model. Inputs shall include predicted speed profiles by the LSTM based driver model, route grade changes, and vehicle specific parameters such as mass, drag coefficient, rolling resistance, and drive-train efficiency. The model shall compute tractive forces, power demand at wheels, motors, battery, energy consumption, and battery state-of-charge (SOC) over the route. For the ease of implementation, if the route exceeds the possible range, the system shall identify the point of SOC depletion and assume battery replenishment at that location. These calculations shall be done before the start of the trip.

**Rationale:** Accurate energy consumption modeling is essential for reliable BEV range estimation. A physics-based approach ensures transparency, interpretability, and adaptability across different vehicle configurations. Remaining range estimation and SOC prediction at the end of the trip are the expected outputs of any BEV range prediction algorithm. These also serve as inputs to any path planning algorithms that use this information to modify the user selected route to include charging station along the route. For the scope of this implementation, we do not include a route planning algorithm.

### Implementation Strategy:

- Implement a backward physics-based model in Python using **NumPy** and **pandas**.
- Calculate battery power, integrate over time calculated from distance and velocity to estimate energy consumption and SOC reduction.
- If the predicted SOC falls below 0%, flag the location of battery exhaustion.
- Use **Matplotlib** for energy consumption and SOC visualization.

## FR-006: Visualization and User Feedback

**Requirement:** The system shall provide an interface to visualize the predicted speed profile, energy consumption, and SOC trajectory. Also provide interfaces that shall allow the developer to adjust vehicle parameters (e.g., mass, HVAC load) and view the impact on energy consumption and SOC estimates.

**Rationale:** Detailed, interactive visualizations of the predicted speeds, energy consumption and SOC depletion ensure transparency and build trust for the user in the system's accuracy. The

parameter tuning interface for the developer are limited to the scope of this project. In a real implementation, these parameters are fixed according to the vehicle and driver settings. But parameter tuning enables the developer to simulate different driving conditions and vehicle configurations making the system more adaptable to different vehicle models.

**Implementation Strategy:**

- Display predicted velocity, acceleration, energy consumption, and SOC progression as interactive subplots using **Matplotlib**
- Integrate optional controls for tuning vehicle parameters such as mass, accessory load, tire rolling resistance coefficient, air density etc using tkinter or other python GUI modules

## **FR-007: Offline Training and Model Management**

**Requirement:** The driver behavior model shall be an offline machine learning model based on Bidirectional LSTM neural network architecture trained using the IEEE Vehicle Speed Dataset available at <https://ieee-dataport.org/open-access/vehicle-speed-dataset> to predict the future vehicle speed for a pre-configured horizon as dictated in FR-004. The training module shall perform data preprocessing, feature engineering, model validation, and checkpoint storage. It shall support retraining or fine-tuning using new driving datasets. It shall also manage versioning of models and associated scalars. The model and the scalars shall be loaded into memory during inference to calculate the energy consumption of the BEV.

**Rationale:** The driver model is trained offline in the context of this project. However, in a real-world implementation, the driver model will be trained online by collecting the driver inputs and the map data during the course of the trip and will continuously evolve as more data is acquired through onboard sensors. This driver model is then used to estimate the BEV range at the start of the trip.

**Implementation Strategy:**

- Implement the model in **Keras** with TensorFlow backend
- Use Pandas and Valhalla API to augment the IEEE dataset with any additional required features for better training and prediction accuracy.
- Set up structured training pipelines in **Google Colab**.
- Version-track model weights through GitHub and store them in Google Cloud Storage and also locally for inferencing at the point of deployment.
- Automate checkpoint management with TensorFlow **ModelCheckpoint** callback and save the model checkpoints in Google Cloud Storage.

## FR-008: Workflow Orchestration

**Requirement:** The system shall coordinate the execution of all modules in a sequential and fault-tolerant manner. Each module shall expose clear interfaces for input/output, and the workflow shall handle any exceptions providing the user with only most required controls.

**Rationale:** A well-orchestrated pipeline ensures system reliability, maintainability, and ease of debugging. It also supports modular testing and future extensibility.

### Implementation Strategy:

- Develop a Python driver script orchestrating module execution.
- Implement error-handling wrappers for API calls and data dependencies.
- Employ logging via Python's `logging` library to ensure traceability.

## 3.3 Non-Functional Requirements and Development Strategies

While functional requirements define what the system does, non-functional requirements (NFRs) determine how well it performs. Each NFR is developed and validated using targeted design strategies and performance metrics that were derived through a systematic analysis of the system architecture, performance goals, and constraints inherent to machine-learning-driven energy estimation.

### NFR-001: Prediction Accuracy

**Requirement:** The driver behavior model shall achieve a reasonable prediction accuracy on the validation datasets based on the chosen training metric. The BEV energy consumption model shall produce range estimates within  $\pm 10\%$  of ground truth when available.

**Rationale:** Accurate predictions are necessary for user trust and practical utility. Errors in speed or energy estimation directly impact range predictions and increase range anxiety for the user.

### Implementation Strategy:

- Use cross-validation on the training dataset implemented using Tensorflow and Scikit utilities to ensure that the model is not overfitting on the training data thereby hurting prediction accuracy.
- Validate the energy consumption model accuracy by comparing the model outputs to data from real driving cycles.

## NFR-002: Response Latency

**Requirement:** The system shall generate a complete energy consumption estimation cycle including speed prediction, energy and SOC computations, and visualization—within 30 seconds. Feature processing shall complete within 2 seconds.

**Rationale:** Users expect near-real-time feedback when selecting routes or adjusting parameters. However, since in this pipeline the retrieval of the node and edge data using Valhalla API is the slowest step, we allow a higher response latency.

### Implementation Strategy:

- Optimize Valhalla API I/O latency by using requests python library.
- Reduce coordinate data processing latency using multi-threading capabilities of the concurrent python library

## NFR-003: Scalability

**Requirement:** The system shall support scaling to larger datasets, more complex LSTM architectures, and additional map features without requiring major architectural changes.

**Rationale:** Future enhancements may include more detailed map attributes, additional vehicle models, or larger training datasets.

### Implementation Strategy:

- Design modular code and interfaces using Tensorflow utilities that enables increasing the number of features used for training and increasing the size of the dataset.
- Parallelize long running preprocessing tasks with multithreading.

## NFR-004: Modularity and Maintainability

**Requirement:** The system shall follow modular design principles, separating map extraction, feature processing, model inference, and energy computation into independent components. Configuration files shall allow easy adjustment of model parameters and vehicle specifications.

**Rationale:** Modularity supports rapid iteration, debugging, and integration of new models or features.

### Implementation Strategy:

- Apply object-oriented design; each major component becomes a class with standardized interfaces.
- Maintain code using Git branches for version control.

### **NFR-005: Interpretability and User Interface Clarity**

**Requirement:** The system shall provide intuitive visualizations of speed profiles, energy consumption, and SOC trajectories. Users shall be able to adjust vehicle parameters and immediately observe the impact on range.

**Rationale:** Clear visual feedback improves user understanding and supports informed decision-making.

#### **Implementation Strategy:**

- Provide GUI based or command-line based arguments for the tunable parameters.
- Create clearly annotated interactive plots using Python's **matplotlib**

### **NFR-006: Robustness and Fault Tolerance**

**Requirement:** The system shall handle missing map features, incomplete datasets, or model inference errors gracefully by providing fallback messages or default behaviors.

**Rationale:** Real-world data sources may be inconsistent such as unmapped areas or missing data from Valhalla APIs. The system must remain stable under such conditions.

#### **Implementation Strategy:**

- Implement try/except logic with custom error messages for failed API or map data requests.
- Conduct negative testing (invalid addresses, unmapped areas, broken JSON) to confirm graceful system recovery.

### **NFR-007: Reproducibility**

**Requirement:** The system shall store model checkpoints and feature scalers to ensure reproducible inference results across environments.

**Rationale:** Reproducibility is essential for research validation and iterative model development.

### Implementation Strategy:

- Record all scalar parameters and model hyperparameters in configuration files.
- Use Git commit hashes linked to model checkpoints ensuring traceability.

## 3.4 System Modules and Interactions

Based on the above outlined system requirements, a modular architecture comprising 7 primary modules where each component performs a distinct function while exchanging structured data with adjacent modules. The modules in the proposed design are the **Map Display Handler**, **Route Processor**, **Velocity Profile Generator**, **PID Controller**, **Driver LSTM Module**, **BEV Energy Consumption Model**, and **Visualization Module** as shown in Figure 3.1. Their responsibilities and interactions are described below.

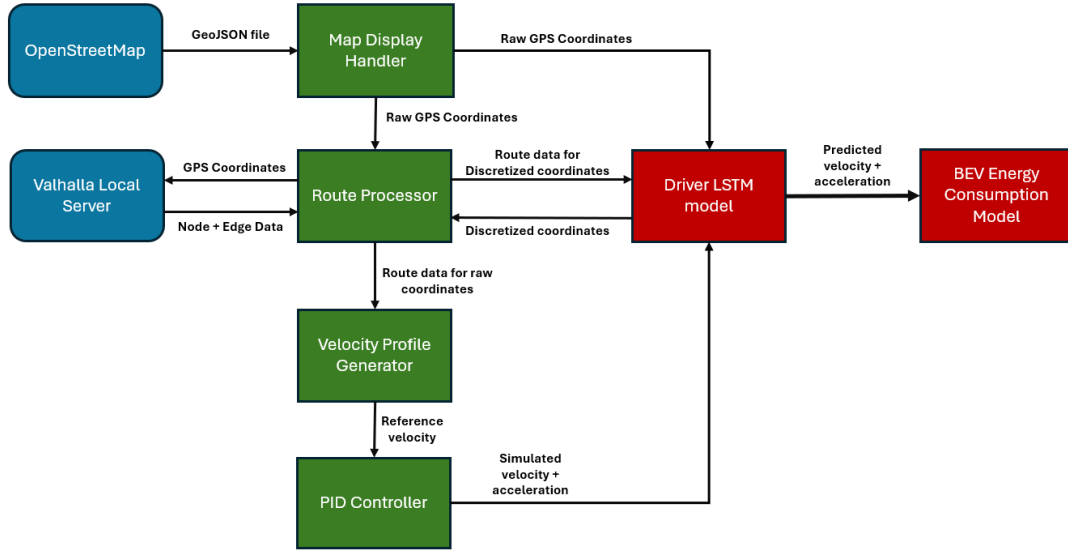


Figure 3.1: Overview of System Architecture

### 1. Map Display Handler

The Map Display Handler serves as the user interface for selecting and retrieving the driving route. It is implemented as a Python–Selenium wrapper around the OpenStreetMap (OSM) web interface, controlled via ChromeDriver. When the user specifies the start and destination points—either by searching addresses or placing markers—the handler monitors changes in the webpage’s URL and interface elements to confirm successful route retrieval. Upon completion, it automatically downloads the corresponding route geometry as a `.geojson` file, containing latitude–longitude coordinates that describe the selected path. The retrieved file becomes the primary

input to the next processing stage.

**Key Functions:**

- Launches OSM within a controlled ChromeDriver session for route selection.
- Detects and stores route changes and metadata automatically.
- Downloads and stores route coordinates in GeoJSON format for reproducibility.

## 2. Route Processor

The Route Processor module transforms raw geographic coordinates into rich, route-specific feature data by querying the Valhalla routing engine through its RESTful APIs. Using the GPS coordinates from the GeoJSON file, it sequentially sends HTTP POST requests to Valhalla's `location` and `elevation` APIs, retrieving node- and edge-level attributes including speed limits, curvature, slope, and intersection details. The results are parsed into a Pandas DataFrame that forms the foundational map feature dataset for downstream modules.

**Key Functions:**

- Interfaces with Valhalla APIs to extract speed limits, curvature, grade, and intersection metadata.
- Performs error-checking for incomplete or inconsistent API responses.
- Produces a structured dataset of all nodes and edges along the route for use in velocity modeling and energy estimation.

## 3. Velocity Profile Generator

The Velocity Profile Generator constructs an initial velocity reference trajectory from node-level data supplied by the Route Processor. This module uses the legal speed limit and the mean edge speed as upper bounds for straight segments, applies lower speed limits for curves and roundabouts, and sets velocity to zero at positions corresponding to traffic lights, stop signs, yield signs, or complex intersections. The generated velocity profile emulates nominal human driving patterns, subject to regulatory and geometric constraints. It provides the reference command input for the PID Controller that follows.

**Key Functions:**

- Translates map node data into a continuous velocity-distance curve.
- Assigns default curve and intersection speed reductions.

- Outputs a route-constrained velocity profile for vehicle kinematics simulation.

#### 4. PID Controller

The PID Controller module models vehicle longitudinal kinematics/dynamics by tracking the reference velocity from the Velocity Profile Generator using a cascaded PID formulation. Through numerical integration of a system of ordinary differential equations, it computes smooth acceleration and jerk-limited transitions, producing a realistic dynamic velocity trajectory that adheres to user-defined constraints. Its output will be used as the past-driving data that will later serve as input for the Driver LSTM Module for batch inferencing.

##### **Key Functions:**

- Simulates longitudinal kinematics constrained by acceleration and jerk limits.
- Generates realistic drive-cycle data representing driver response to map conditions.
- Supplies filtered velocity, acceleration, and positional states for future ML inference.

#### 5. Driver LSTM Module

The Driver LSTM Module is a bidirectional LSTM network based personalized driver-behavior model. It discretizes the route into uniformly spaced 1 meter intervals and retrieves node attributes for the discretized coordinates using the Route Processor module again. The model employs batch inferencing to predict the driver-specific future velocity profile by combining the velocity profile generated by the PID Controller which serves as the past driving data with both past and future road features from the node data, followed by Butterworth and zero-phase filtering to remove high-frequency noise and smooth the predictions. The final output is a refined, driver-specific, map-aware speed profile along the route.

##### **Key Functions:**

- Performs spatial discretization of the route for fine-grained uniformly spaced feature retrieval required for inferencing.
- Conducts batch inference using pre-trained LSTM weights to predict future velocities.
- Applies digital filtering (Butterworth, zero-phase) to produce a physically consistent, noise-free velocity profile.

#### 6. BEV Energy Consumption Model

The BEV Energy Consumption Model converts the predicted velocity profile into energy metrics by simulating tractive forces and power usage through a physics-based backward calculation



approach. It computes instantaneous acceleration, wheel tractive force, aerodynamic drag, rolling resistance, and grade forces to determine mechanical power demand. From these, motor power, battery power, and wheel power consumption across the route are derived. The model then integrates these quantities to obtain total energy consumption and SOC depletion.

#### **Key Functions:**

- Calculates acceleration and tractive power using the predicted velocity profile.
- Estimates cumulative energy and SOC variation across the chosen route.
- Outputs time-series data for power at wheels, motor, and battery subsystems.

## **7. Visualization Module**

The Visualization Module provides graphical representations of all computed vehicle dynamics and energy metrics, facilitating analysis and performance evaluation. It plots the predicted and reference velocity profiles against route distance with overlaid road feature markers (e.g., intersections, grade changes, or speed-limit zones). It also displays instantaneous power consumption at the wheels, motor, and battery, total energy usage, and estimated SOC trajectory.

#### **Key Functions:**

- Visualizes speed-distance and power-distance profiles with annotated map features.
- Depicts cumulative energy consumption and SOC change.
- Provides comparative plots of commanded, simulated, and predicted velocities for validation.

## **Module Interaction and Data Flow**

Figure 3.2 summarizes the interactions among all seven modules. The overall data flow can be described as follows:

1. The **Map Display Handler** enables route selection on OSM and stores route geometry in a GeoJSON file.
2. The **Route Processor** reads this file and retrieves node attributes, producing a structured dataset of map features.
3. The **Velocity Profile Generator** converts these node features into an initial reference velocity profile, defining legal and nominal speed constraints along the route.
4. The **PID Controller** uses this reference velocity as a commanded input to simulate real-world vehicle response, generating dynamic time-series data that emulate actual driving.

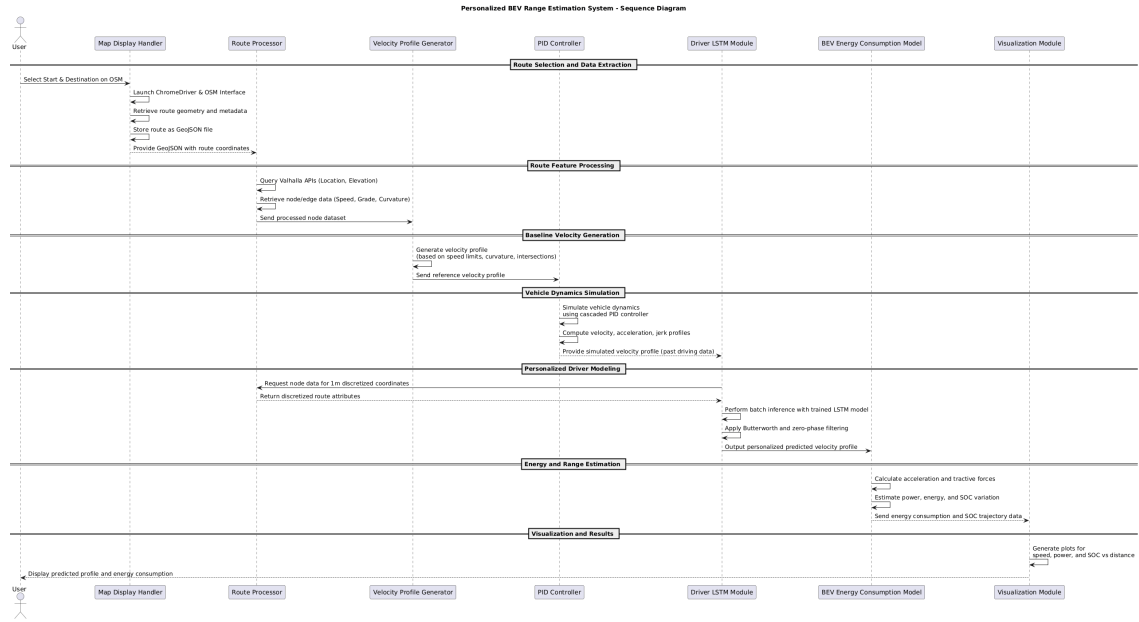


Figure 3.2: Data flow and interaction among system modules.

5. The **Driver LSTM Module** takes the discretized route features and the simulated past-driving profile to infer driver-specific future velocities, generating a personalized predicted speed profile.
6. The **BEV Energy Consumption Model** processes this predicted profile to compute acceleration, tractive power, battery load, and overall energy usage—producing the route-level range and SOC projections.
7. Finally, the **Visualization Module** aggregates and plots the results, showing velocity, power, and SOC variations aligned with route geometry for interpretability.

## Chapter 4

# System Design and Implementation

This chapter presents the design and implementation of the personalized Battery Electric Vehicle (BEV) energy estimation framework that integrates driver behavior modeling with map-based contextual data. It explains the theoretical foundations and practical development of each system module, detailing how they collectively enable accurate, personalized, and route-aware range prediction.

The framework is structured into seven key modules: Map Display Handler, Route Processor, Velocity Profile Generator, PID Controller, Driver LSTM Module, BEV Energy Consumption Model, and Visualization Module. Each module is designed to perform a dedicated function—ranging from route data extraction and driver-specific speed prediction to vehicle kinematics simulation and energy consumption estimation—while maintaining seamless data exchange across the pipeline. The sections that follow describe the theoretical principles and algorithmic logic underlying each component, followed by the implementation details using the selected technology stack.

### 4.1 Map Display Handler

The Map Display handler module, as previously mentioned, is a Python Selenium wrapper designed for interacting with the OpenStreetMap.org website at <https://www.openstreetmap.org/> [22]. The website interaction is facilitated using ChromeDriver.

ChromeDriver [12] is a standalone server that implements the W3C WebDriver and WebDriver BiDi standards. WebDriver is an open source tool for automated testing of webapps across many browsers. It provides capabilities for navigating to web pages, user input, JavaScript execution, and more. ChromeDriver extends Webdriver by adding Chromium-specific capabilities. It uses the **ChromeOptions** object to pass capabilities to ChromeDriver from the WebDriver API. ChromeDriver enables the user to programmatically control the Chrome browser.

Python Selenium [8] is a powerful library for automating web browser interactions. It enables developers to write scripts that simulate user actions, making it useful for web scraping, testing, and automating repetitive tasks. ChromeDriver acts as a bridge between Selenium and the Chrome browser, allowing automated interactions like clicking buttons, filling forms, and navigating web pages.

In our implementation, the Map Display Handler module is the first to run when the user starts the program. Selenium configures **ChromeOptions** and initiates ChromeDriver, which launches the Chrome browser with the webpage <https://www.openstreetmap.org/directions>. The user can then select the desired route by entering source and destination addresses or by placing the source and destination markers. Additionally, they can choose the routing engine—Valhalla, Grasshopper, or OSRM—for navigation. Once the route and engine are selected and the user presses enter, the chosen routing engine generates the directions.

In the background, Selenium continuously monitors the website for URL changes which occur whenever the user interacts with the website by clicking buttons or entering text using event listeners. A successful retrieval of directions is confirmed by detecting specific HTML elements on the page. If the URL changes due to user interaction but the expected elements are absent, then a successful retrieval of directions is not confirmed and Selenium continues polling for further URL updates. This process continues for a timeout period of 120 seconds. Each time the user interacts with the webpage, the timeout resets back to 120 seconds. However, if no interaction occurs within this period, the session expires, prompting Selenium to automatically close the ChromeDriver instance and terminate the program.

Once a successful retrieval of directions is verified through the presence of specific HTML elements, Selenium automatically clicks the "Download route as GeoJSON" element to download the GPS coordinates along the route as a JSON file. This process occurs each time the URL changes and the directions are successfully retrieved. An example of the Chrome browser operated by Selenium after a successful retrieval of directions is shown in Figure 4.1. An example of the GeoJSON file downloaded by Selenium that contains the GPS coordinates of the selected route is shown in Figure 4.2

## 4.2 Route Processor

The Route Processor module extracts and processes the node and edge data for the GPS coordinates retrieved by the Map Display Handler corresponding to the user-defined route. The module executes immediately after the Map Display Handler. It checks for the existence of the GeoJSON file in the current working directory and if the file is missing, the module raises an exception and terminates the program. However, if the file is found, it proceeds to extract the GPS coordinates

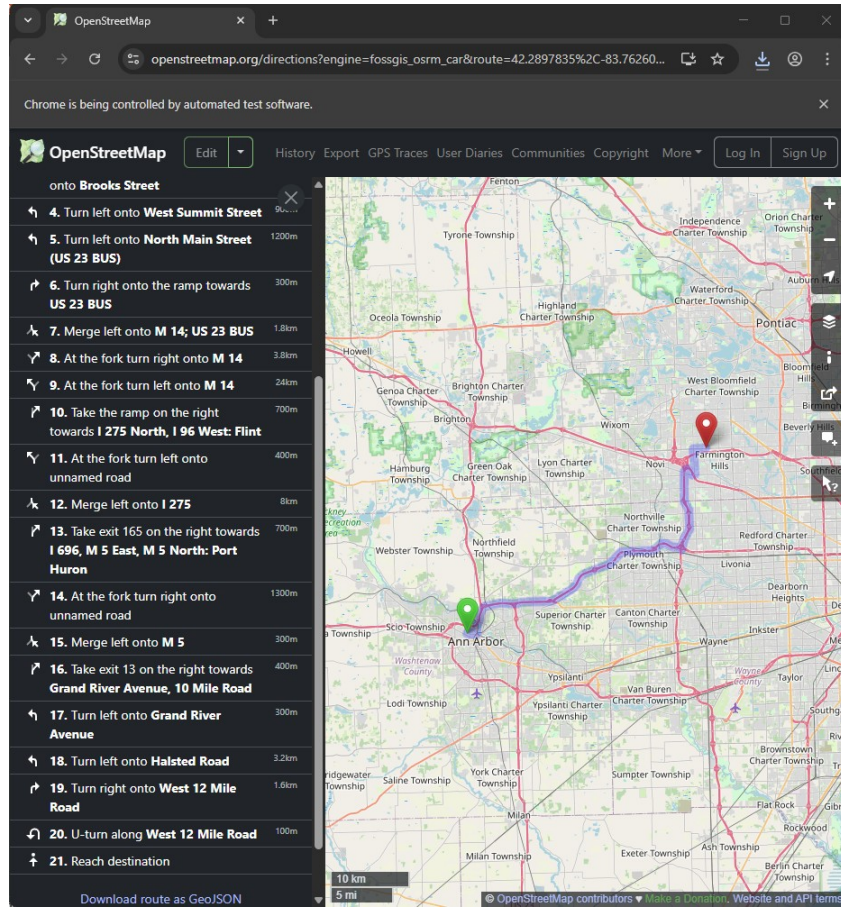


Figure 4.1: Chrome browser controlled by Selenium showing the OpenStreetMap.org webpage after a successful retrieval of directions



Figure 4.2: GeoJSON file containing the coordinates of the selected route

for the route. It then retrieves the node and edge data for these coordinates by utilizing Valhalla routing engine’s APIs.

Valhalla [9] [10] is an open-source routing engine designed for use with OpenStreetMap (OSM) data. It provides highly customizable and efficient routing solutions, supporting various transportation modes such as driving, cycling, and walking. Valhalla is known for its tiled hierarchical data structure, which enables efficient routing, offline capabilities, and regional extracts.

Valhalla offers several APIs that allow developers to integrate routing and navigation functionalities into applications described below :

- **Routing API:** This API provides turn-by-turn directions for various transportation modes, including driving, cycling, and walking. Users can specify locations, costing models, and output preferences, and the API returns detailed route information such as maneuvers, distances, and estimated travel times.
- **Isochrone API:** This API generates travel time polygons, allowing users to visualize areas that can be reached within a specified time frame. It is particularly useful for accessibility analysis and service coverage mapping.
- **Map Matching API:** This API refines GPS traces by aligning them with road networks, improving accuracy for tracking applications and historical route analysis. It ensures that recorded GPS data corresponds to actual roads.
- **Elevation API:** This API provides elevation data for specified locations, making it useful for terrain-aware routing and outdoor activity planning. It helps users understand elevation changes along a route.
- **Matrix API:** This API calculates travel times and distances between multiple locations, supporting logistics and delivery optimization. It enables efficient route planning for scenarios involving multiple destinations.
- **Optimized Route API:** This API solves the Traveling Salesman Problem (TSP), allowing users to plan the most efficient multi-stop routes. It is ideal for delivery services and fleet management, ensuring minimal travel time and distance.
- **Location API:** This API retrieves detailed information about streets and intersections near a given point, helping applications determine the closest road segments and improve navigation accuracy. While it does not perform geocoding, it can be integrated with external services to convert addresses into coordinates for routing purposes.

These Valhalla APIs are accessed through RESTful POST requests to a locally hosted instance of the Valhalla routing engine, deployed via Docker on the localhost at port 8002. In this configuration, the local Valhalla server provides significantly faster response times and is not constrained

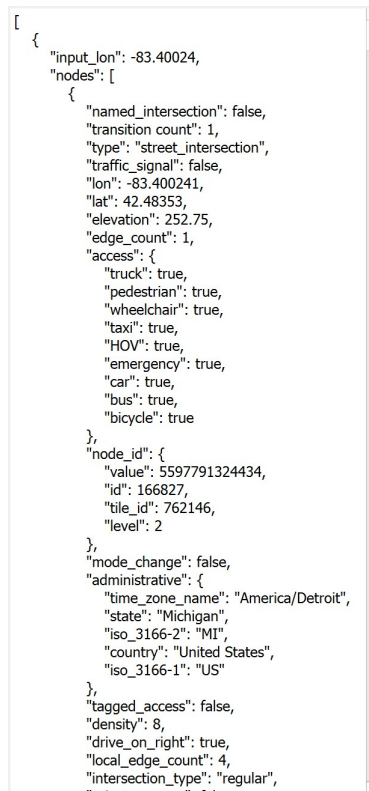


by request-rate limits, enabling efficient retrieval of node and edge data corresponding to the GPS coordinates extracted from the GeoJSON file. The publicly available Valhalla instance hosted at <https://valhalla1.openstreetmap.de> is used as a fallback if the local server is unavailable or incomplete data is retrieved from the local server.

The module processes the GPS coordinates extracted from the GeoJSON file in chunks using multi-threading for parallelization, generating RESTful requests to the Valhalla APIs with these coordinates as input. During each iteration, a subset of GPS coordinates is selected and incorporated into the HTTP POST request payload sent to the Valhalla server. The payload is URL encoded and the transmission of the request is managed through the cURL tool. An example of the URL encoded cURL command for the Location API is:

```
curl "http://localhost:8002/locate?json=%7B%22locations%22%3A%5B%7B%22lat%22%3A42.28321%2C%22lon%22%3A-83.74841%7D%5D%2C%22costing%22%3A%22auto%22%2C%22verbose%22%3Atrue%7D"
```

Upon receiving the response from the Valhalla API, the payload is decoded from URL encoding into a structured JSON format for further processing. An example of the decoded payload from the Valhalla Location API response in JSON format is shown in Figure 4.3. The flow of the Route Processor module for one iteration is shown in Figure 4.4



```
[
  {
    "input_lon": -83.40024,
    "nodes": [
      {
        "named_intersection": false,
        "transition count": 1,
        "type": "street_intersection",
        "traffic_signal": false,
        "lon": -83.400241,
        "lat": 42.48353,
        "elevation": 252.75,
        "edge_count": 1,
        "access": {
          "truck": true,
          "pedestrian": true,
          "wheelchair": true,
          "taxi": true,
          "HOV": true,
          "emergency": true,
          "car": true,
          "bus": true,
          "bicycle": true
        },
        "node_id": {
          "value": 5597791324434,
          "id": 166827,
          "tile_id": 762146,
          "level": 2
        },
        "mode_change": false,
        "administrative": {
          "time_zone_name": "America/Detroit",
          "state": "Michigan",
          "iso_3166-2": "MI",
          "country": "United States",
          "iso_3166-1": "US"
        },
        "tagged_access": false,
        "density": 8,
        "drive_on_right": true,
        "local_edge_count": 4,
        "intersection_type": "regular",
        "named_intersection": false
      }
    ]
  }
]
```

Figure 4.3: Example of the Valhalla Location API response payload in JSON format





The compiled node data consists of the following fields :

- Latitude : Latitude of the node.
- Longitude : Longitude of the node.
- Range : The range is the absolute distance in meters up to the node from the starting node.
- Elevation : The elevation of the node above the sea level in meters
- Edge ID : A unique identifier of an edge to which the node belongs within the tiled, hierarchical graph.
- Percent along Edge : Location of the node along its corresponding edge in percentage. 0% indicates the node is located at the start of the edge and 100% indicates that the node is located at end of the edge.
- Speed limit : Speed limit in *km/h* at the node
- Edge Speed : Average speed along the edge in *km/h*
- Heading : Heading angle at the node ranging from  $0^\circ$  to  $360^\circ$  with  $0^\circ$  corresponding to North,  $90^\circ$  corresponding to East,  $180^\circ$  corresponding to South and  $270^\circ$  corresponding to West.
- Edge Mean Elevation : The mean or average elevation in meters above the sea level along the edge. A value of 32768 indicates no elevation data is available for this edge.
- Curvature : A curvature factor ranging from 0 to 15, where higher values indicate a smaller radius of curvature and consequently a lower average speed.
- Edge Max Upward grade : The maximum upward slope in %. A value of 32768 indicates no elevation data is available for this edge.
- Edge Max Downward grade : The maximum downward slope in %. A value of 32768 indicates no elevation data is available for this edge.
- Edge Weighted grade : A weighted grade factor for the edge. Valhalla manufactures a weighted grade from elevation data. It is a measure used for hill avoidance in routing – sort of a relative energy use along an edge. But since an edge in Valhalla can possibly go up and down over several hills it might not equate to what most people think of as grade.
- Traffic Signal : A flag indicating whether a traffic signal is present at the node.
- Stop Sign : A flag indicating whether a stop sign is present at the node.
- Yield Sign : A flag indicating whether a yield sign is present at the node.
- Round About : A flag indicating whether the node is part of a round-about

- **Edge Classification** : An enumeration which represents the hierarchical importance of a road-way segment, where lower numeric values indicate higher-order roads—specifically: motorway(0), trunk(1), primary(2), secondary(3), tertiary(4), unclassified(5), residential(6), and service/other(7)
- **Edge Link** : A flag indicating short connector segments, such as ramps or transition links, that connect different road classes within the routing network.
- **Intersection Type** : Intersection type at the node which can be of the values – street intersection, gate, bollard, toll booth, multi-use transit stop, bike share, parking, motorway junction, border control.
- **Intersection Complexity** : A description assigned to the intersection based on the type and number of edges intersecting at the node.
- **Way ID** : Way identifier of the OpenStreetMap base data.

### 4.3 Velocity Profile Generator

The Velocity Profile Generator constructs a reference velocity profile based on the node and edge data compiled by the Route Processor module. This reference profile is subsequently utilized by the PID controller module to generate point-to-point command velocities as vehicle kinematics are simulated.

The current implementation of the Velocity profile Generator iterates through all the nodes and generates the reference velocity profile according to the following conditions :

1. If a traffic signal, stop sign, or yield sign is detected at the node, the reference velocity is set to  $0\text{ km/h}$ , and the speed limit type is assigned as 3.
2. If the node is part of an intersection and located at 0% along the edge and the heading angle change from the previous edge to the current edge exceeds  $60^\circ$ , the reference velocity is set to  $0\text{ km/h}$ , with a speed limit type of 3.
3. If the node is part of an intersection and located at 100% along the edge and the heading angle change from the current edge to the next edge exceeds  $60^\circ$ , the reference velocity is set to  $0\text{ km/h}$ , with a speed limit type of 3.
4. If the curvature factor at a node is greater than a calibrated threshold of 5, then the reference velocity is set as the minimum of the speed calculated based on lateral acceleration constraints and the average speed at the node retrieved from the Location API response. The speed limit type is set to 2

5. If none of the above overriding conditions apply, the reference velocity at the node is set to the speed limit retrieved from the Location API response, with a speed limit type of 1.

The speed limit type serves as an additional parameter that informs the PID controller module how to generate the command velocity based on the characteristics of the road being traversed. A speed limit type of 2 signifies a curved road segment, requiring velocity adjustments for safe traversal. A speed limit type of 3 designates a stop node, indicating that the vehicle must come to a complete halt. Conversely, a speed limit type of 1 implies that the specified speed limit can be directly used as the command velocity without any modifications. This structured approach ensures that velocity adjustments align with critical roadway attributes, intersection dynamics, and curvature constraints, contributing to accurate vehicle movement simulations.

It is important to note that a reference speed of  $0\text{ km/h}$  and a speed limit type of 3 have been assigned to all intersections where the heading angle change exceeds  $60^\circ$ , requiring the vehicle to come to a complete stop for all right and left turns. However, this does not universally apply to all intersections. For instance, when transitioning from a main roadway to a residential street via a right turn, the vehicle generally does not stop completely but rather slows down to approximately  $30\text{ km/h}$ . Nevertheless, since the primary objective is energy consumption estimation, modeling the worst-case scenario—where the vehicle comes to a full stop at intersections with a heading angle change greater than  $60^\circ$ —provides a conservative estimate for its electric range.

## 4.4 PID Controller

The PID controller module simulates the longitudinal vehicle dynamics that adhere to the user-defined constraints by utilizing the reference velocity profile constructed by the Velocity Profile Generator module as the control input. It does this by iteratively integrating a system of ordinary differential equations until the entire reference velocity profile data has been processed. It is composed of two submodules: the Command Velocity Generator and the Vehicle Dynamics Model. The Command Velocity Generator utilizes the reference velocity profile along with the current vehicle states to compute the commanded velocity at each time step of simulation. The Vehicle Dynamics Model then uses this commanded velocity as the control input and predicts the vehicle's states – distance, velocity, acceleration, and jerk – at the next step by solving a system of ordinary differential equations. A detailed discussion of each of these submodules follows in the subsequent sections. An overview of the PID controller module that depicts the interaction of these two submodules is shown in Figure [4.6](#)

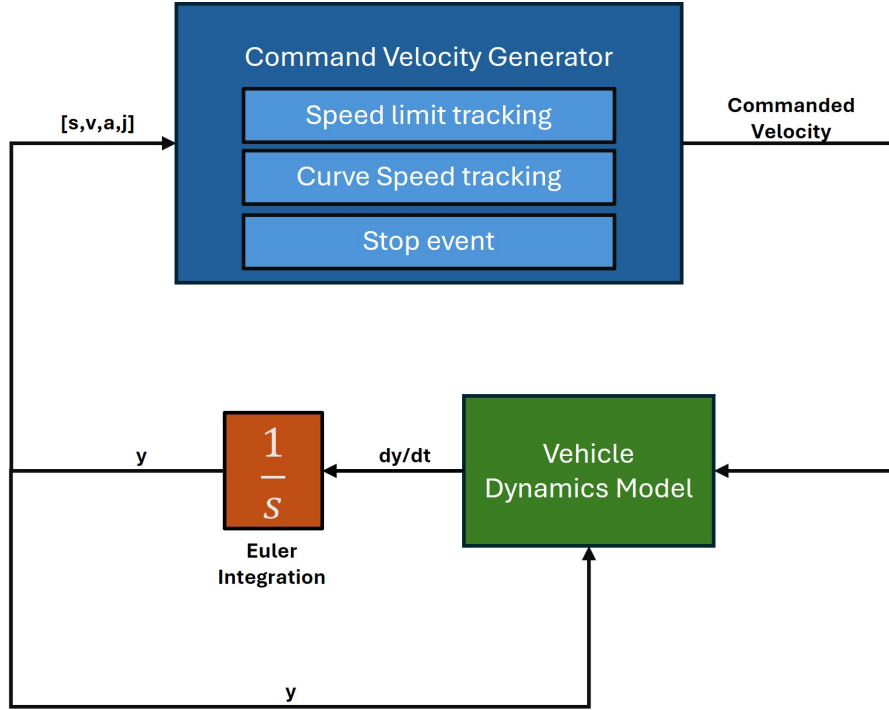


Figure 4.6: Overview of the PID Controller Module

#### 4.4.1 Command Velocity Generator

The Command Velocity Generator calculates the commanded velocity at each time step of the integration by adjusting the reference velocity profile from the Velocity Profile Generator module according to the speed limit type. This adjustment is necessary to account for events such as curve speed tracking and stop events. In other words, while the Velocity Profile Generator module produces a static reference speed trajectory from the node data, the Command Velocity Generator submodule dynamically adapts this reference speed based on the vehicle kinematics. In this way, the Command Velocity Generator submodule is an extension of the Velocity Profile Generator module. However, it has been integrated into the PID controller module since its execution occurs within the vehicle kinematics simulation loop.

For standard speed limit tracking of speed limit type 1, no adjustments to the reference speed profile are necessary, as the driver naturally adapts vehicle speed following a change in the posted speed limit. However, in the case of curved road segments, the driver begins braking before reaching the curve to ensure that the vehicle remains within its lateral traction limits. Consequently, for speed limit type 2, the command velocity must be reduced before entering the curve event, allowing the vehicle to reach the required curve speed before the event begins. A similar argument applies to stop events, where the driver decelerates prior to reaching a zero-speed node, such as an intersection or stop sign. Therefore, for speed limit type 3, the command velocity must be set to zero ahead of the stop event to ensure that the vehicle comes to a complete halt at the designated point.

## Algorithm of the Command Velocity Generator

The Command Velocity Generator submodule is designed as a state machine that takes the current vehicle states – distance, velocity, acceleration and jerk – as the inputs and produces the command velocity as the output. The command velocity is determined by the current state of the state machine. Note the state of the state machine is different from the vehicle states – distance, velocity, acceleration and jerk . The Command Velocity Generator contains 3 states :

- **Standard Speed Limit tracking :** The Standard speed limit tracking is the default state of the state machine. It sets the command velocity as the speed limit from the reference velocity profile at the current distance traveled by the vehicle. It also determines the conditions to transition to other states by calculating a look-ahead horizon  $s_h(t)$
- **Curve speed tracking :** The curve speed tracking state is triggered when a curved road segment of speed limit type 2 appears in the look-ahead horizon  $s_h(t)$  of the state machine. It determines the command velocity based on the distance to the curve event and the distance required to attain the curve speed. Once the curve speed event has been completed or if a stop event has been detected in the look-ahead horizon  $s_h(t)$ , the state transitions back to Standard Speed Limit tracking.
- **Stop event tracking :** The stop event tracking state is triggered when a stop event of speed limit type 3 has been detected in the look-ahead horizon  $s_h(t)$ . It sets the command velocity to  $0m/s$  based on the current acceleration of the vehicle, the distance to the stop event and the distance required to attain zero speed. Once the stop event is completed, the state transitions back to Standard Speed Limit tracking.

The Command Velocity Generator makes use of pre-calculated look-up tables to determine the stopping distances  $s_s(t)$  for a particular velocity and acceleration. These stopping distance for velocities ranging from  $0 - 200 km/h$  are determined by pre-simulation of the vehicle kinematics. In these pre-simulations, the initial vehicle velocity is set to the velocity for which the stopping distance is to be calculated and the commanded velocity is set as  $0 m/s$  throughout the simulation. Then the distance required for the vehicle to come to a stop while adhering to the vehicle kinematics constraints is determined from the simulation. This process can be repeated for various initial acceleration values to generate a 2-D lookup table for the stopping distances with one axis as the initial velocity and the other axis as the initial acceleration. However, we have used two 1-D lookup tables in our implementation – one for initial acceleration of  $0 m/s^2$  and another for initial acceleration of  $2 m/s^2$  – which will be referred to as zero acceleration LUT,  $LUT_{a0}(v)$  and max acceleration LUT,  $LUT_{amax}(v)$  respectively.

The overview of the Command Velocity Generator module with the different states is shown in Figure 4.7. A brief description of the algorithm of the Command Velocity Generator state machine is as follows :

1. In order to calculate the command speed  $v_c(t)$  at the current time step  $t$ , the distance traversed  $s(t)$  by the vehicle is determined from the vehicle states. The reference speed  $v_l(s(t))$  at the

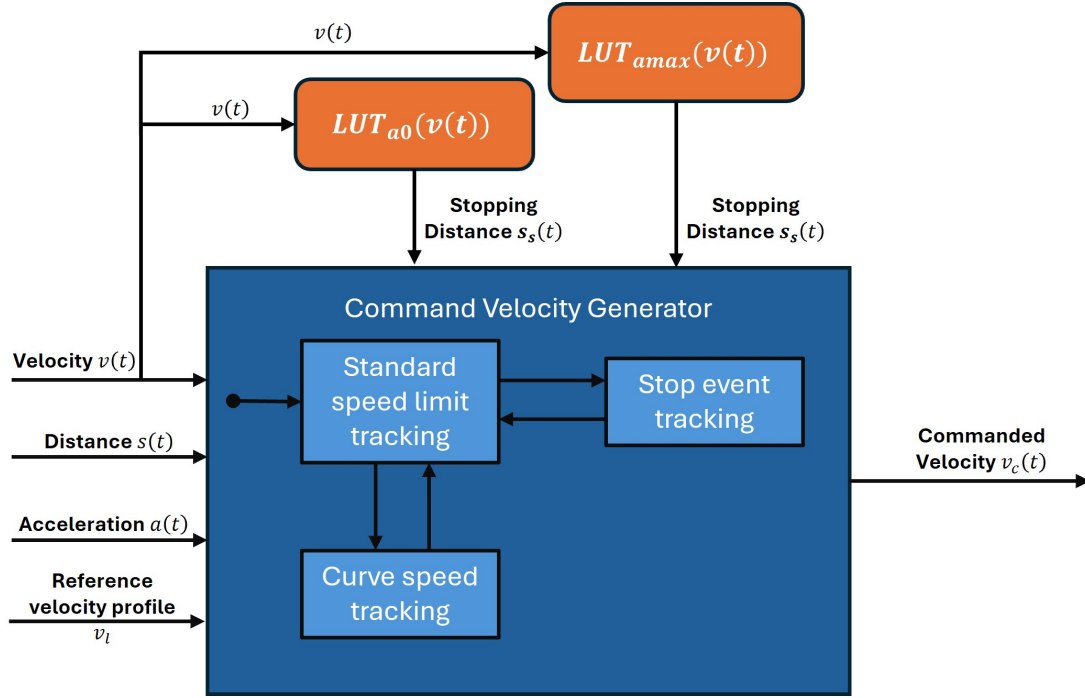


Figure 4.7: Overview of the Command Velocity Generator submodule

current distance  $s(t)$  from the reference velocity profile generated by the Velocity Profile Generator is determined.

2. Additionally, the look-ahead horizon  $s_h(t)$  is calculated at each time step based on the stopping distance for the current vehicle velocity  $v(t)$ . The look-ahead horizon is calculated from the max acceleration LUT, as this is the worst-case scenario where more distance is required for stopping as below,

$$s_h(t) = LUT_{amax}(v(t))$$

3. Then all the future speed limit change events occurring within the look-ahead horizon  $s_h(t)$  are identified. The distance to the future event,  $s_f(t) - s(t)$  from the current point and distance required to attain the future speed  $s_r(v(t), a(t), v_f(t))$  are determined. Here  $s_f(t)$  is the absolute distance of the future event and  $v_f(t)$  is the reference vehicle velocity at the future event.

$$v_f(t) = v_l(s_f(t))$$

If the future event is of speed limit type 3, the distance required to attain the future speed is the stopping distance determined from the max acceleration LUT as,

$$s_r(t) = s_s(t) = LUT_{amax}(v(t))$$

If the future event is of speed limit type 2, then the distance required to attain the future

speed is the stopping distance determined from the max acceleration LUT multiplied by the ratio of speed change to current speed as,

$$s_r(t) = LUT_{amax}(v(t)) \left( \frac{v(t) - v_f(t)}{v(t)} \right) = s_s(t) \left( \frac{v(t) - v_f(t)}{v(t)} \right)$$

4. If the current distance to the future speed limit change event,  $s_f(t) - s(t)$  is less than the distance required to attain future speed  $s_r(t)$  i.e  $s_f(t) - s(t) < s_r(t)$ , then we transition to Curve Speed tracking state if the future speed limit type,  $vtyp(s_f)$  is 2. We transition to the Stop event tracking state if the future speed limit type,  $vtyp(s_f)$  is 3. If there are no future speed limit change events which require immediate modification of our commanded speed i.e.  $s_f(t) - s(t) > s_r(t)$ , then we use the current speed limit  $v_l(s(t))$  as our commanded speed  $v_c(t)$  as,

$$v_c(t) = v_l(s(t))$$

5. If the Stop event tracking state has been triggered, we recalculate the stopping distance  $s_s(t)$  to stop event based on current acceleration  $a(t)$ . We use the max acceleration LUT,  $LUT_{amax}(v(t))$  if we have an acceleration  $a(t) > 0.5 \text{ m/s}^2$ , else we use the zero acceleration LUT  $LUT_{a0}(v(t))$  to calculate the stopping distance. If the current distance to the stop event,  $s_f(t) - s(t)$  is less than the distance  $s_s(t)$  required to attain  $0\text{m/s}$  i.e.  $s_f(t) - s(t) < s_s(t)$ , we set the commanded speed  $v_c(t)$  as  $0\text{m/s}$  until the stop point has been reached with zero velocity as,

$$v_c(t) = 0$$

Once the vehicle comes to stop, the speed limit after the stop event is set as the commanded speed as,

$$v_c(t) = v_l(s_f(t)^+)$$

( where  $v_l(s_f(t)^+)$  is speed limit right after the stop event )

And the Stop event tracking state is transitioned back to Standard speed limit tracking state.

6. If the curve-speed tracking logic has been triggered, we set the commanded velocity as curve speed limit until the curve speed is attained. This commanded velocity is reduced below the curve speed limit if distance to the curve event reduces faster than the distance to required to attain the curve speed according to following equation,

$$v_c(t) = v_f(t) - \left| v_f(t) \tanh \left( \frac{k s_r(t)}{s_f(t) - s(t)} \right) \right|$$

This enables the PID controller to increase the deceleration of the vehicle in order to attain the curve speed before the curve event. Once the curve event is crossed, the curve-speed

tracking state is terminated and transitioned to the Standard speed limit tracking state.

In the curve-speed tracking logic, the look-ahead horizon  $s_h(t)$  is always repopulated to check if there are any upcoming stop events or more curve events which require further deceleration. If there are upcoming events which require further deceleration, then the curve-speed tracking state is terminated and control is returned to the standard speed limit tracking state to repopulate the look-ahead horizon  $s_h(t)$  and decide the next state transition.

#### 4.4.2 Vehicle Dynamics Model

The vehicle dynamics model submodule models the vehicle system as a simple PID controller with tunable gains and constraints on the vehicle states. The default values of the constraints on the vehicle states are decided according to the requirements of driving comfort and are as follows :

- **Acceleration constraints :** Maximum of  $+2m/s^2$  acceleration and Minimum of  $-4m/s^2$  deceleration
- **Jerk constraints :** Maximum of  $+10m/s^3$  jerk and Minimum of  $-10m/s^3$  jerk

The system of ordinary differential equations that define the PID Controller are given in equation (4.1) as follows :

$$\begin{aligned}\frac{ds}{dt} &= v \\ \frac{dv}{dt} &= a \\ \frac{da}{dt} &= j \\ \frac{dj}{dt} &= k_3 (\text{sat}([k_2 (\text{sat}([k_1 (v_c - v)] ; a_{max}, a_{min}) - a)] ; j_{max}, j_{min}) - j)\end{aligned}\tag{4.1}$$

where,

$$\text{sat}(x; x_{min}, x_{max}) = \begin{cases} x_{max} & x \geq x_{max} \\ x & x_{min} < x < x_{max} \\ x_{min} & x \leq x_{min} \end{cases}$$

The function  $\text{sat}(x; x_{min}, x_{max})$  imposes the maximum and minimum constraints,  $x_{max}$  and  $x_{min}$  respectively, on a state variable  $x(t)$ .

In the system of ODEs defined by Equation (4.1),  $\{s(t), v(t), a(t), j(t)\}$  are the vehicle states — distance, velocity, acceleration and jerk — at time  $t$  respectively.

$a_{max}$  and  $a_{min}$  are maximum and minimum constraints respectively on the acceleration  $a(t)$ .

$j_{max}$  and  $j_{min}$  are maximum and minimum constraints respectively on the jerk  $j(t)$ .

The tunable gains of the PID controller,  $k_1, k_2, k_3$  from the system of ODEs defined by Equation (4.1) can be selected to emulate the desired behavior of the vehicle states. Generally, for stable



performance of the PID controller that minimizes the overshoot in higher order states, the gains are selected such that  $k_1 < k_2 < k_3$ . A diagrammatic representation of the system of ODEs, Equation (4.1) is shown in Figure 4.8, where both the Command Velocity Generator submodule and the Vehicle Dynamics Model submodule are distinctly highlighted.

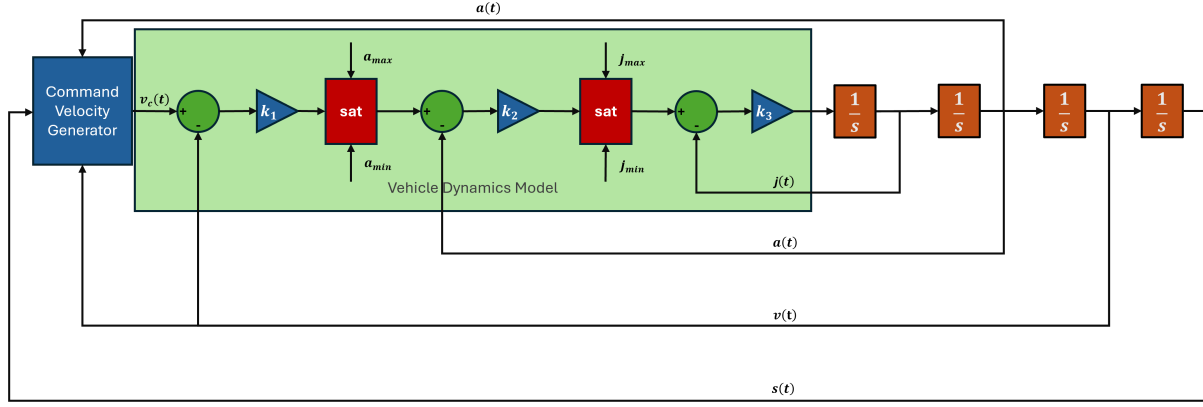


Figure 4.8: Diagrammatic representation of Vehicle Dynamics model

## Euler's Method

To solve the system of ODEs defined by Equation (4.1) any numerical ODE solver may be used. The Runge-Kutta methods are preferred due to their ability to minimize local truncation errors at each timestep. However, given the simplicity and computational efficiency of Euler's method, it is utilized in this implementation with a very small timestep  $h$  to reduce the truncation errors. The definition of the Euler method is as follows :

Consider a system of ordinary differential equations defined by the equation (4.2)

$$\frac{dy}{dt} = \underline{f}(t, \underline{y}) \quad (4.2)$$

where  $\underline{y} = \{y_1 \ y_2 \ \dots \ y_n\}^T \in \mathbb{R}^n$  is the state vector and  $\underline{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$  is nonlinear vector function. The underline indicates that the quantity is a vector.

The system of ordinary differential equations defined by Equation (4.2) can be integrated from time  $t = 0$  to determine the states  $\underline{y}(t)$  at any future time  $t$  when the initial condition  $\underline{y}(0)$  is known. This is known as the initial value problem and can be solved using Euler's method according to Equation (4.3),

$$\underline{y}(t + h) = \underline{y}(t) + \underline{f}(t, \underline{y}) h \quad (4.3)$$

where  $h$  is the timestep of integration. The local truncation error of the Euler Method is of order  $O(h^2)$

Using the above described framework, the PID controller module simulates the vehicle kinematics and generates a vehicle velocity profile for the user-selected route. Once the reference velocity profile data is fully processed, the simulation concludes, and the resulting velocity, acceleration, and

jerk profiles are supplied to the Driver LSTM module. This vehicle kinematics information provided by the PID controller serves as the past driving input sequence for the Driver LSTM model which produces a personalized velocity profile. This integration enables batch inferencing during the LSTM evaluation, accelerating computation while refining the simulated velocity profile from the PID Controller with driver-specific dynamic characteristics.

## 4.5 Driver LSTM Module

The Driver LSTM Module is designed to capture the temporal dependencies between a driver’s past actions and the road geometry—both historical and upcoming—to produce accurate forecasts of future vehicle dynamics. The module forms the machine learning component of the personalized BEV range estimation framework, extending the deterministic vehicle dynamics produced by the PID Controller with driver-specific behavioural characteristics. This is achieved through a sequence-to-sequence (seq2seq) neural network architecture that exploits the temporal recurrence property of Long Short-Term Memory (LSTM) networks and their bidirectional variants.

### 4.5.1 Long Short-Term Memory Networks

LSTM networks form the backbone of the prediction strategy due to their ability to model long-range temporal dependencies while mitigating the vanishing-gradient problem that affects traditional recurrent neural networks (RNNs). Each LSTM cell maintains two internal states: a hidden state and a cell state. The cell state acts as a long-term memory buffer, while the hidden state captures short-term dynamics. A diagrammatic representation of an LSTM cell is shown in Figure 4.9

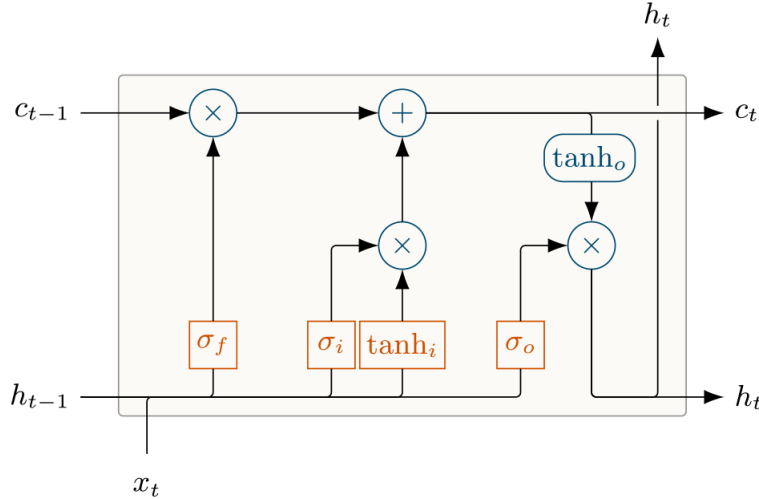


Figure 4.9: Diagrammatic representation of an LSTM cell

An LSTM cell operates through three gating mechanisms:

1. **Forget Gate:** The forget gate determines which information from the previous cell state should be discarded. The equation for the forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (4.4)$$

where  $\sigma()$  is the sigmoid activation function which maps a real-valued input into the range  $[0, 1]$ .  $W_f$  and  $b_f$  are the weights and biases of the forget gate used to calculate the forget factor  $f_t$  respectively.  $[h_{t-1}, x_t]$  denotes the concatenation of the input at the current timestep  $t$  and the hidden state at the previous timestep  $t - 1$

2. **Input Gate:** The Input gate controls how much new information should be written to the cell state. The equation for the input gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (4.5)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (4.6)$$

where  $\tanh()$  is the hyperbolic tangent activation function that maps a real-valued input into the range  $[-1, 1]$ .  $W_i$  and  $b_i$  are the weights and biases used to calculate the input factor  $i_t$  which controls how much the new information candidate cell  $\tilde{c}_t$  is to be added to the previous cell state  $c_{t-1}$ .  $W_c$  and  $b_c$  are the weights and biases used to calculate the candidate cell state  $\tilde{c}_t$ .

3. **Output Gate:** The output gate is responsible for deciding what part of the current cell state should be sent as the hidden state (output) for this time step. The equation for the output gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (4.7)$$

where  $W_o$  and  $b_o$  are the weights and biases used to calculate the output factor  $o_t$  of the output gate.

The updated cell and hidden states for the current time step are then computed as :

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (4.8)$$

$$h_t = o_t \odot \tanh(c_t) \quad (4.9)$$

This gating structure allows LSTMs to selectively retain or discard information, making them particularly effective for modelling driver behaviour, which often depends on both immediate actions and longer-term behavioural patterns.

### 4.5.2 Bidirectional LSTM Networks

Bidirectional LSTMs extend the standard LSTM by processing the input sequence in both forward and backward directions. For each time step, the BiLSTM produces two hidden states: forward LSTM output and backward LSTM output. These are concatenated to form a richer representation.

In the context of this project, BiLSTMs are used because of their ability to incorporate information from the entire input sequence—both past and future relative to each time step—allowing the model to form a more complete understanding of the temporal structure of the data. For example, the past driving behavior encoder benefits from recognizing patterns that unfold symmetrically around certain events (e.g., braking before a curve), while the future road feature encoder benefits from understanding the global structure of the upcoming road segment.

### 4.5.3 Prediction Framework

The prediction strategy developed adapts the LSTM architecture proposed by Paparusso. et. al.[24] to the specific requirements of this project. The objective is to construct a model that jointly encodes past driver-vehicle behavior, contextualize it with respect to both the past and future road geometry, and recursively generate multi-step forecasts of future vehicle states. This problem can be formally formulated as follows.

Let  $k \in \mathbb{N}$  denote the current position index. We consider a fixed-length history window of past vehicle positions of size  $s_P \in \mathbb{N}$  and a prediction horizon of future vehicle positions of size  $s_F \in \mathbb{N}$ . We define:

- A **past-vehicle state matrix**  $X_{(k-s_P:k)}^{vP} \in \mathbb{R}^{(s_P+1) \times n_v}$  where  $n_v$  is the size of the vehicle states that are used for prediction. These vehicle states are physical quantities representing vehicle kinematics such as linear velocity, angular velocity, linear acceleration, angular acceleration, yaw rate etc. Therefore  $X_{(k-s_P:k)}^{vP}$  will be a matrix such as:

$$X_{(k-s_P:k)}^{vP} = \begin{bmatrix} v_{k-s_P}^1 & v_{k-s_P}^2 & \dots & v_{k-s_P}^{n_v} \\ v_{k-s_P+1}^1 & v_{k-s_P+1}^2 & \dots & v_{k-s_P+1}^{n_v} \\ \dots & \dots & \dots & \dots \\ v_k^1 & v_k^2 & \dots & v_k^{n_v} \end{bmatrix} \quad (4.10)$$

where  $v_i^j$  represents the  $j$ th vehicle state at  $i$ th position.

- A **past-road context matrix**  $X_{(k-s_P:k)}^{rP} \in \mathbb{R}^{(s_P+1) \times n_r}$  containing road features corresponding to the same indices of the past-vehicle state matrix  $X_{(k-s_P:k)}^{vP}$ . These road features include quantities that define the road segment characteristics such as legal speed limit, curvature, grade, road class (motorway, primary, service etc), presence of traffic sign, stop sign, yield

sign etc. Therefore  $X_{(k-s_P:k)}^{rP}$  will be a matrix such as:

$$X_{(k-s_P:k)}^{rP} = \begin{bmatrix} r_{k-s_P}^1 & r_{k-s_P}^2 & \cdots & r_{k-s_P}^{n_r} \\ r_{k-s_P+1}^1 & r_{k-s_P+1}^2 & \cdots & r_{k-s_P+1}^{n_r} \\ \cdots & \cdots & \cdots & \cdots \\ r_k^1 & r_k^2 & \cdots & r_k^{n_r} \end{bmatrix} \quad (4.11)$$

where  $r_i^j$  represents the  $j$ th road feature at  $i$ th position.

- A **future-road context matrix**  $X_{(k+1:k+s_F)}^{rF} \in \mathbb{R}^{s_F \times n_r}$  describing the road geometry ahead which will be a matrix such as:

$$X_{(k+1:k+s_F)}^{rF} = \begin{bmatrix} r_{k+1}^1 & r_{k+1}^2 & \cdots & r_{k+1}^{n_r} \\ r_{k+2}^1 & r_{k+2}^2 & \cdots & r_{k+2}^{n_r} \\ \cdots & \cdots & \cdots & \cdots \\ r_{k+s_F}^1 & r_{k+s_F}^2 & \cdots & r_{k+s_F}^{n_r} \end{bmatrix} \quad (4.12)$$

- The future vehicle-state matrix  $Y_{(k+1:k+s_F)}^{vF} \in \mathbb{R}^{s_F \times q_v}$ , where  $q_v \leq n_v$  is the future vehicle states which can be a subset of all the vehicle states for the future position interval of  $[k+1, k+s_F]$ . The future vehicle-state matrix  $Y_{(k+1:k+s_F)}^{vF}$  will be a matrix such as:

$$Y_{(k+1:k+s_F)}^{vF} = \begin{bmatrix} v_{k+1}^1 & v_{k+1}^2 & \cdots & v_{k+1}^{q_v} \\ v_{k+2}^1 & v_{k+2}^2 & \cdots & v_{k+2}^{q_v} \\ \cdots & \cdots & \cdots & \cdots \\ v_{k+s_F}^1 & v_{k+s_F}^2 & \cdots & v_{k+s_F}^{q_v} \end{bmatrix} \quad (4.13)$$

The prediction target is the future vehicle-state matrix  $Y_{(k+1:k+s_F)}^{vF}$ . In other words, the goal is forecast the evolution of a subset of vehicle states over the next  $s_F$  positions. The model seeks to approximate the nonlinear mapping:

$$f_\theta : \left( X_{(k-s_P:k)}^{vP}, X_{(k-s_P:k)}^{rP}, X_{(k+1:k+s_F)}^{rF} \right) \mapsto Y_{(k+1:k+s_F)}^{vF}$$

where  $\theta$  denotes the network parameters.

#### 4.5.4 Proposed Architecture

To solve the above described problem, a hybrid encoder–decoder architecture based on Long Short-Term Memory (LSTM) and Bidirectional LSTM (BiLSTM) networks, augmented with a autoregressive decoding mechanism that enables generation of temporally dependent sequences of predicted states is proposed. The strategy is motivated by the observation that driver–vehicle dynamics are inherently sequential, nonlinear, and strongly conditioned by both historical behavior informed by past road context and the structure of the future road ahead. The proposed framework

therefore integrates these 3 sources of information: a past-driving encoder that summarizes the recent evolution of the vehicle states, a past-road encoder that captures the dependence of past vehicle states on the past road geometry and a future-road encoder that extracts contextual cues from the upcoming road geometry. These encoded representations are fused and projected into a latent state that seeds a recursive decoder responsible for generating the future trajectory of the system.

The proposed hybrid encoder–decoder architecture for driver-specific future vehicle state prediction shown in Figure 4.10 is as follows:

1. The past vehicle state matrix  $X_{(k-s_P:k)}^{vP}$  and the past road context matrix  $X_{(k-s_P:k)}^{rP}$  are concatenated along the feature axis and are encoded using a bidirectional LSTM layer with  $u_e$  hidden units, for their high performance in sequence encoding. The last hidden states  $h_F^P$ ,  $h_B^P$  and cell states  $c_F^P$ ,  $c_B^P$  of the BiLSTM layers in the forward and backward directions are merged together through concatenation, to form one hidden states vector  $h^P$  and one cell states vector  $c^P$  representing the past driving encoding.
2. The future road context matrix  $X_{(k+1:k+s_F)}^{rF}$  is encoded by means of another bidirectional LSTM layer with  $u_e$  hidden units, employing the same merge mechanism, to obtain the hidden states vector  $h_F$  and cell states vector  $c_F$  representing the future road encoding.
3. The hidden states and cell states of the two encoders are then concatenated, to create an extended hidden states vector  $h^{PF} = [h^P, h_F]$  and an extended cell states vector  $c^{PF} = [c^P, c_F]$
4. Two dense layers with  $2u_e$  and  $u_d$  neurons are applied to both  $h^{PF}$  and  $c^{PF}$  without weights sharing, to merge the information carried by the past driving and future road encoding, and bring map them to the dimensions of the decoder. A dropout layer with rate  $r$  is placed between two dense layers to promote regularization. Consequently, the vectors  $h^D$  and  $c^D$  are generated.
5. The decoder consists of two stages. The first is a unidirectional LSTM layer with  $u_d$  hidden units, whose hidden and cell states are initialized using the vectors  $h^D$  and  $c^D$ . The first cell is fed with  $h^D$  as inputs. From the second cell on, the output of each LSTM cell is recursively used as input of the successive cell.
6. The second stage of the decoder is a bidirectional LSTM layer of  $u_d$  hidden units, whose output sequences are fed to a final time-distributed dense layer, that brings the matrices to assume the output shape. The output of the network  $\hat{Y}_{(k+1:k+s_F)}^{vF}$  is the prediction of  $Y_{(k+1:k+s_F)}^{vF}$ , which represents the ground truth.

This encoder–decoder mechanism allows the model to capture short- and long-term dependencies while contextualizing the driver’s dynamic response to future road geometry.

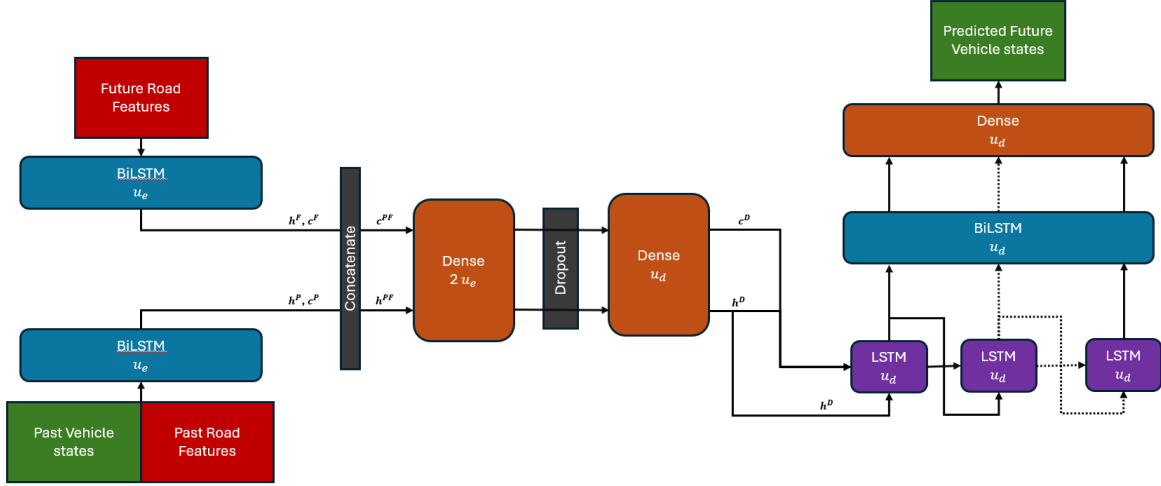


Figure 4.10: Overview of the Driver LSTM Module Encoder-Decoder Architecture

#### 4.5.5 Implemented Architecture

Based on the proposed design, the implemented architecture incorporates both the past vehicle states and past and future road features as follows:

- **Past vehicle states:** The vehicles states used in the past vehicle state matrix  $X_{(k-s_P:k)}^{vP}$  are:
  - Longitudinal velocity
  - Longitudinal acceleration
- **Past and future road features:** The road features used in the past and future road context matrices  $X_{(k-s_P:k)}^{rP}$  and  $X_{(k+1:k+s_F)}^{rF}$  are:
  - Elevation
  - Heading
  - Speed Limit
  - Average Edge Speed
  - Curvature
  - Traffic Signal
  - Stop Sign
  - Yield Sign
  - Round About
  - Edge Classification
  - Edge Link
  - Start Stop

As can be seen all these road features except 'Start Stop' are obtained from the node data compiled by the Route Processor module. The Start Stop is an engineered feature which is a flag to indicate that the vehicle is starting from a standstill at the beginning of a sequence or coming to standstill at the end of a sequence as indicated in the dataset. Of all road features and vehicle states, numerical variables such as elevation, heading, speed limit, Average Edge Speed, Curvature, Longitudinal velocity and Longitudinal acceleration are scaled using SciPy's `StandardScaler`.

Inputs are structured using a past window of  $s_P = 50$  steps and a future prediction horizon of  $s_F = 100$  steps. The encoder employs a Bidirectional LSTM with a hidden size  $u_e = 32$ , and the decoder uses a unidirectional LSTM with hidden size  $u_d = 64$ . A dropout layer with rate  $r = 0.25$  is applied for regularization between the dense layers.

#### 4.5.6 Training the Driver LSTM Model

The IEEE Vehicle Speed dataset available at <https://ieee-dataport.org/open-access/vehicle-speed-dataset>[31] was used for the training of the implemented architecture. The dataset contains 5973 rides with a total length of 9049.3 km. The data was collected during 2021 in the Czech Republic and includes municipal and non-municipal trips. The dataset contains uniformly spaced data with 1 meter spatial intervals. The data contains the features shown in Table 4.1 in addition to other engineered features. A snapshot of the dataset is shown in Figure 4.11.

During preliminary data exploration, it became evident that the original dataset contained numerous inconsistencies, particularly within the categorical attributes, where schema definitions and value formats were not applied uniformly. Due to the extent of these anomalies, any categorical variables from the original dataset were excluded from the training process. Instead, the road features—described in subsection 4.5.5—were generated using the Route Processor module applied to the GPS coordinates provided in the dataset. These derived features were then appended to the original dataset, and the resulting augmented dataset was finally used for the training.

Table 4.1: Summary of Route Geometry and Node-Level Features

| Name         | Type        | Description                                                                                        |
|--------------|-------------|----------------------------------------------------------------------------------------------------|
| target_speed | continuous  | Actual vehicle speed                                                                               |
| fwd_azimuth  | continuous  | Forward azimuth or heading                                                                         |
| elevation    | continuous  | Absolute elevation.                                                                                |
| way_maxspeed | continuous  | Speed limit extracted from OpenStreetMap (OSM).                                                    |
| way_surface  | categorical | OSM road surface type (e.g., asphalt, sett, cobblestone).                                          |
| way_type     | categorical | OSM road classification (e.g., primary, highway, residential).                                     |
| node         | categorical | Node-specific features such as level crossings, traffic signs, intersections, and traffic signals. |

The encoder-decoder architecture described in subsection 4.5.5 was implemented and trained in Keras using the Adam optimizer with a learning rate of 0.0001 and the mean-squared error loss function. Prior to training, all numerical variables were transformed using scalers fitted on the complete dataset. The dataset was then partitioned into training, validation, and testing subsets



| latitude    | longitude   | target_speed | way_maxspeed | speed_orm   | elevation   | hwd_azimuth  | way_type   | way_surface | node:intersection | node:railway | node:crossing | node:highway    | node:stop | start_stop | azimuth_diff | elevation_diff |
|-------------|-------------|--------------|--------------|-------------|-------------|--------------|------------|-------------|-------------------|--------------|---------------|-----------------|-----------|------------|--------------|----------------|
| 49.194054   | 16.569149   | 14.459415    | 50           | 1.598039215 | 208         | 89.21939432  | tertiary   | asphalt     |                   |              |               |                 |           | 1          | 0            | 0              |
| 49.19405412 | 16.56916272 | 14.459415    | 50           | 2.184657204 | 208.0612436 | 89.21939432  | tertiary   | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405424 | 16.56917644 | 14.459415    | 50           | 2.771275192 | 208.1224873 | 89.21939432  | tertiary   | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405437 | 16.56919016 | 14.459415    | 50           | 3.35789318  | 208.1837309 | 89.21939432  | tertiary   | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405449 | 16.56920387 | 14.459415    | 50           | 3.944511169 | 208.2449746 | 89.21939432  | tertiary   | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405461 | 16.56921759 | 14.459415    | 50           | 4.531129157 | 208.3062182 | 89.21939432  | tertiary   | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405473 | 16.56923131 | 14.459415    | 50           | 5.117747147 | 208.3674618 | 89.21939432  | tertiary   | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405486 | 16.56924503 | 14.459415    | 50           | 5.704365133 | 208.4287055 | 89.21939432  | tertiary   | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405498 | 16.56925875 | 14.459415    | 50           | 6.290983122 | 208.4899491 | 89.21939432  | tertiary   | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.1940551  | 16.56927247 | 14.459415    | 50           | 6.87760111  | 208.5511927 | 89.21939432  | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405522 | 16.56928619 | 14.459415    | 50           | 7.464219098 | 208.6124364 | 89.21939432  | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405535 | 16.5692999  | 14.459415    | 50           | 8.050837096 | 208.67368   | 89.21939432  | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405547 | 16.56931362 | 14.459415    | 50           | 8.637455074 | 208.7349237 | 89.21939432  | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405559 | 16.56932734 | 14.459415    | 50           | 9.224073063 | 208.7961673 | 89.21939432  | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405571 | 16.56934106 | 14.459415    | 50           | 9.810691051 | 208.8574109 | 89.21939432  | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405584 | 16.56935478 | 14.459415    | 50           | 10.39739904 | 208.9186546 | 89.21939432  | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | 0.061243638    |
| 49.19405596 | 16.5693685  | 14.459415    | 50           | 10.98392703 | 208.9798962 | 89.21939432  | trunk_link | asphalt     | side_to_main      |              |               | traffic_signals |           | 0          | 113.9633564  | -0.015290529   |
| 49.19406149 | 16.56936914 | 14.459415    | 50           | 11.17647059 | 208.9646077 | -24.74396211 | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | -0.05268489    |
| 49.19406965 | 16.5693634  | 14.459415    | 50           | 11.17647059 | 208.9119228 | -24.74396211 | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | -0.05268489    |
| 49.19407782 | 16.56935766 | 14.459415    | 50           | 11.17647059 | 208.8592379 | -24.74396211 | trunk_link | asphalt     |                   |              |               |                 |           | 0          | 0            | -0.05268489    |

Figure 4.11: Training of the Driver LSTM Model

following a 70:15:15 ratio. Input sequences were generated using a stride of 10 steps along with pre-padding and post-padding, and these sequences were subsequently organized into batches for both training and validation. Model training was conducted for 30 epochs, with validation loss monitored to mitigate overfitting through early stopping, and model weights were checkpointed regularly to enable retraining. The retraining was done multiple times with randomization of input data to improve the performance of the model on the defined metric : Mean Absolute error. The evolution of the loss function in one of the training session is shown in Figure 4.12



Figure 4.12: Overview of the Command Velocity Generator submodule

## 4.5.7 Inference Procedure

After training, the model with the best testing performance is deployed locally along with its corresponding feature scalars. To efficiently generate the predicted driver-specific velocity profile,

a batch inferencing strategy is used rather than a fully recursive multi-step forecasting approach. This strategy leverages the simulated velocity profile generated by the PID controller as the representation of past vehicle velocity, enabling faster inference.

To prepare the inputs for inference, the route extracted from the GeoJSON file is discretized into uniformly spaced GPS coordinates at 1-meter intervals. The Route Processor Module is then used to retrieve node and edge attributes for each discretized coordinate. The simulated velocity and acceleration profiles are interpolated onto these coordinates to form the past vehicle state input features. These data points are subsequently processed, using a stride of 10 steps, into batches of sequences containing past vehicle states  $X^{vP}$  and , past road-context features  $X^{rP}$ , and future road-context features  $X^{rF}$ . These batches of past and future feature matrices are fed into the loaded LSTM model to predict the future velocity profile  $\hat{Y}^{vF}$ , from which only the first 10 predicted steps are retained for each inference window.

The predicted velocity segments are then post-processed using a first-order Butterworth low-pass filter [6], which smooths high-frequency fluctuations while preserving the underlying behavioral trends. A zero-phase forward-backward filtering [7] step is subsequently applied to eliminate noise without introducing phase delay or distorting the shape of the filtered signal.

The resulting filtered predictions form the final driver-specific velocity profile, capturing the natural driving behavior learned by the model. This predicted profile is then passed to the BEV Energy Consumption Model for downstream estimation of tractive power demand, energy usage, and SOC evolution along the route.

## 4.6 BEV Energy Consumption Model

Accurately estimating the energy consumption and battery State of Charge (SOC) of a Battery Electric Vehicle (BEV) requires a physics-based representation of the forces acting on the vehicle, the power conversion processes within the electric powertrain, and the mechanisms governing battery charge depletion and regeneration. The modeling framework adopted in this thesis follows the quasi-steady, backward approach introduced in the VT-CPEM framework described by Fiori et al.[11]. As stated in the paper, the model “computes an EV’s instantaneous energy consumption using second-by-second vehicle speed, acceleration and roadway grade data as input variables,” enabling high-resolution estimation of tractive power, motor power, battery power, and SOC evolution along the route. This section presents the theoretical foundations of the model, including the computation of wheel power, motor power, battery power, energy consumption, and SOC evolution.

#### 4.6.1 Tractive Power

The starting point of the backward formulation is the computation of the tractive power required at the wheels. At each timestep, the total tractive force is computed as the sum of inertial, grade, rolling-resistance, and aerodynamic components:

$$F_{\text{inertia}}(t) = m a(t) \quad (4.14)$$

$$F_{\text{grade}}(t) = mg \sin(\theta(t)) \quad (4.15)$$

$$F_{\text{roll}}(t) = mg (C_2 + C_1 v(t)) \frac{C_r}{1000} \quad (4.16)$$

$$F_{\text{aero}}(t) = \frac{1}{2} \rho A_f C_D v(t)^2 \quad (4.17)$$

$$(4.18)$$

The total force is:

$$F_{\text{total}}(t) = F_{\text{inertia}}(t) + F_{\text{grade}}(t) + F_{\text{roll}}(t) + F_{\text{aero}}(t) \quad (4.19)$$

The instantaneous power at the wheels is then:

$$P_{\text{wheels}}(t) = F_{\text{total}}(t) v(t) \quad (4.20)$$

Positive values correspond to traction, while negative values correspond to braking.

where,  $m$  is the vehicle mass,  $v$  is the vehicle longitudinal velocity,  $a$  is the vehicle longitudinal acceleration,  $g$  is gravitational acceleration,  $\theta$  is the road grade angle,  $C_r$  is the rolling resistance coefficient,  $C_1$  and  $C_2$  are empirical constants,  $\rho$  is the air density,  $A_f$  is the frontal area and  $C_D$  is the aerodynamic drag coefficient

#### 4.6.2 Electric Motor Power

The electric motor power is obtained by adjusting the wheel power using fixed driveline and motor efficiencies, consistent with the VT-CPEM assumptions. During traction, the motor supplies power to the wheels; during braking, the power flow reverses.

During traction ( $P_{\text{wheels}} \geq 0$ ), the motor power is:

$$P_{\text{motor}}(t) = \frac{P_{\text{wheels}}(t)}{\eta_{\text{driveline}} \eta_{\text{motor}}} \quad (4.21)$$

During braking ( $P_{\text{wheels}} < 0$ ), the motor operates in generator mode. The regenerative braking efficiency  $\eta_{\text{rb}}$  is modeled as an exponential function of deceleration:

$$\eta_{rb}(t) = \begin{cases} e^{-\left(\frac{\alpha}{|a(t)|}\right)} & \forall a(t) < 0 \\ 0 & \forall a(t) \geq 0 \end{cases} \quad (4.22)$$

with the constraint:  $0 \leq \eta_{rb}(t) \leq \eta_{rb,max}$

The recovered motor power is then:

$$P_{motor}(t) = P_{wheels}(t) \eta_{rb}(t) \eta_{driveline} \eta_{motor} \quad (4.23)$$

where  $\eta_{driveline}$  is the driveline efficiency and  $\eta_{motor}$  is the motor efficiency

### 4.6.3 Battery Power and Auxiliary Loads

The net battery power accounts for motor power, auxiliary loads  $P_{aux}$  (e.g., HVAC, electronics), and battery efficiency  $\eta_{batt}$ : For traction:

$$P_{batt}(t) = \frac{P_{motor}(t) + P_{aux}}{\eta_{batt}} \quad (4.24)$$

For Regeneration:

$$P_{batt}(t) = (P_{motor}(t) + P_{aux}) \eta_{batt} \quad (4.25)$$

The sign convention ensures that positive battery power corresponds to discharge and negative power corresponds to charging.

### 4.6.4 Energy Consumption Estimation

The energy consumption over a route segment is computed by integrating the instantaneous battery power over time:

$$E(t) = \int P_{batt}(t) dt \quad (4.26)$$

And final energy consumption per kilometer in Wh/km is computed as:

$$EC = \frac{1}{3600} \int P_{batt}(t) dt \cdot \frac{1}{d} \quad (4.27)$$

where  $d$  is the total distance traveled in meters.

#### 4.6.5 SOC Dynamics

The battery State of Charge  $SOC$  is updated using:

$$\Delta SOC(t) = \frac{P_{\text{batt}}(t)}{3600 \cdot C_W} \quad (4.28)$$

$$SOC(t) = SOC(t-1) - \Delta SOC(t) \quad (4.29)$$

where  $C_W$  is the battery capacity in Wh

To preserve battery health, SOC is constrained to remain above 20% and below 95% which forms the operating range of the battery

#### 4.6.6 Integration with Driver-Specific Velocity Predictions

The final driver-specific velocity profile generated by the LSTM inference pipeline is fed into the VT-CPEM model proposed in the above sections to compute the instantaneous tractive power, regenerative braking potential, energy consumption and SOC evolution. Because the model is highly sensitive to transient speed and acceleration behavior, it is well-suited for capturing the influence of individual driving styles on energy consumption. This enables personalized range estimation that reflects both route characteristics and driver behavior

## Chapter 5

# Testing and Results

### 5.1 Development of Unit Test Cases

A structured and comprehensive set of unit test cases was developed to validate each module of the personalized BEV range-estimation system. The goal of these test cases is to ensure that every subsystem—ranging from map data extraction to vehicle dynamics and energy modeling—functions correctly in isolation. The test cases were derived directly from the functional requirements, expected behavior and outputs, and failure modes outlined in the system design.

#### 5.1.1 Map Display Handler Test cases

Test cases for the Map Display Handler Test Cases were designed to verify robustness, user interaction handling, and error recovery.

- **ChromeDriver Initialization:** Validate that the ChromeDriver launches correctly and loads the OpenStreetMap directions interface.
- **Route Selection and GeoJSON Retrieval:** Confirm that selecting a route — either through address input or marker placement — produces a valid GeoJSON file containing the route coordinates.
- **Unexpected Closure Handling:** Ensure the system detects missing or corrupted GeoJSON files and the user is prompted to reselect the route or exit gracefully.
- **User Timeout:** Verify automatic termination of the ChromeDriver after 2 minutes of inactivity, with timeout reset on every user interaction.
- **Route Editing:** Confirm that the system retains only the most recent route selection when the user edits a route and discards outdated selections.

### 5.1.2 Route Processor Test Cases

Test cases for the Route Processor module focus on data completeness, consistency, and integration of multiple API responses.

- **Coordinates Processing:** Validate that all coordinates in the GeoJSON file are processed using the Valhalla routing engine and are populated with node and edge data when using multi-threading.
- **API Integration Consistency:** Ensure that outputs from the Locate, Map Matching, and Elevation APIs are matched and merged properly without discontinuities in heading, geometry, or edge alignment while processing in chunks using multi-threading.
- **Handling Missing Node Data:** Confirm that missing or unpopulated node attributes are filled using information from adjacent nodes or edges to ensure consistency and continuity of data.
- **Timeout and Error Handling:** Test system behavior when the local Valhalla server instance times out, returns empty data or becomes unreachable. Ensure that the fallback to the open-source server <https://valhalla1.openstreetmap.de> is working.

### 5.1.3 Velocity Profile Generator Test Cases

Test cases for Velocity Profile Generator module validate that the module applies the correct rule hierarchy and produces the correct reference velocities and speed limit type

- **Traffic Sign, stop sign, yield sign handling:** Verify that the reference velocity is set to 0 km/h when a traffic signal, stop sign, or yield sign is detected at the node, and that the speed-limit type is correctly assigned as 3.
- **Intersection Stop Logic:** Ensure that the module also sets a 0 km/h reference speed and speed-limit type 3 at intersections where the heading angle change exceeds 60° coupled with the start or end of an edge.
- **Curvature-Based Speed Reduction:** Confirm that when the curvature factor exceeds the threshold of 5, the reference velocity is reduced to the minimum of the lateral acceleration limited speed and the API-reported average edge speed, with speed-limit type 2.
- **Default Speed Limit Tracking:** Validate that when no other overriding conditions apply, the reference velocity follows the legal speed limit retrieved from the Location API, with speed-limit type 1.
- **Handling Missing Speed Data:** Verify that in case of missing legal speed limit, the average edge speed is used and in case of both missing node and edge data, the previous reference speed and speed limit type are used.

### 5.1.4 PID Controller Test Cases

The Test cases for PID Controller module validate the correctness of speed commands for different speed limit types in the horizon, enforcement of acceleration and jerk constraints and integration stability.

- **Legal Speed Limit Tracking:** Verify that the commanded velocity does not exceed posted speed limits.
- **Curve Negotiation:** Ensure that the model reduces speed appropriately before entering curves event in the driver-lookup horizon to prevent excessive deceleration before the curve.
- **Stop event tracking:** Verify that the model commands a velocity profile that avoids excessive braking for every stop and the vehicle velocity attains 0 km/h before the node with the stop sign, traffic light, yield sign, or intersections with large heading change.
- **Constraint Enforcement:** Validate that acceleration and jerk for the generated velocity profile remain within the calibrated constraints.
- **Numerical Stability:** Verify that the numerical error of integration does not accumulate for long-duration simulations.

### 5.1.5 Driver LSTM Module Test Cases

The test cases for the Driver LSTM Module concentrate on the core reliability and correctness of the inference pipeline.

- **Sequence Construction and Scaling:** Verify that the past vehicle states, past road-context features, and future road-context features are correctly assembled into fixed-length sequences using the required stride, and that all numerical variables are transformed using the fitted scalers from training.
- **Batch Inferencing Consistency:** Ensure that the batch inferencing pipeline correctly processes the discretized route data, interpolated velocity inputs, and sliding-window sequences without misalignment or data leakage across batches.
- **Prediction Stability and Continuity:** Confirm that the LSTM model produces stable and continuous velocity predictions across successive inference windows, and that the concatenated predictions do not exhibit discontinuities, spikes, or unrealistic transitions.
- **Filtering and Post-Processing Validation:** Validate that the predicted velocity segments are correctly smoothed using the first-order Butterworth low-pass filter followed by zero-phase forward-backward filtering, ensuring removal of high-frequency noise without introducing phase distortion.



### 5.1.6 BEV Energy Consumption and SOC Estimation Module Test Cases

Test cases for the BEV Power Consumption Model focus on the correctness and alignment with published benchmarks for standard drive cycles.

- **Standard Drive Cycle Validation:** Evaluate the implemented model against published values of Energy consumption / km (Wh/km) in [11] for the following standard drive cycles:
  - Worldwide harmonized Light-duty vehicles Test Cycle (WLTC)
  - World Motorcycle Test Cycle (WMTC)

The validation of the BEV Energy model on the WLTC cycle is shown in Figure 5.1. The calculated energy consumption for the WLTC cycle from our BEV Energy Consumption model with 700 W auxiliary load is  $169.89 \text{ Wh/km}$  which matches with the value published in [11].

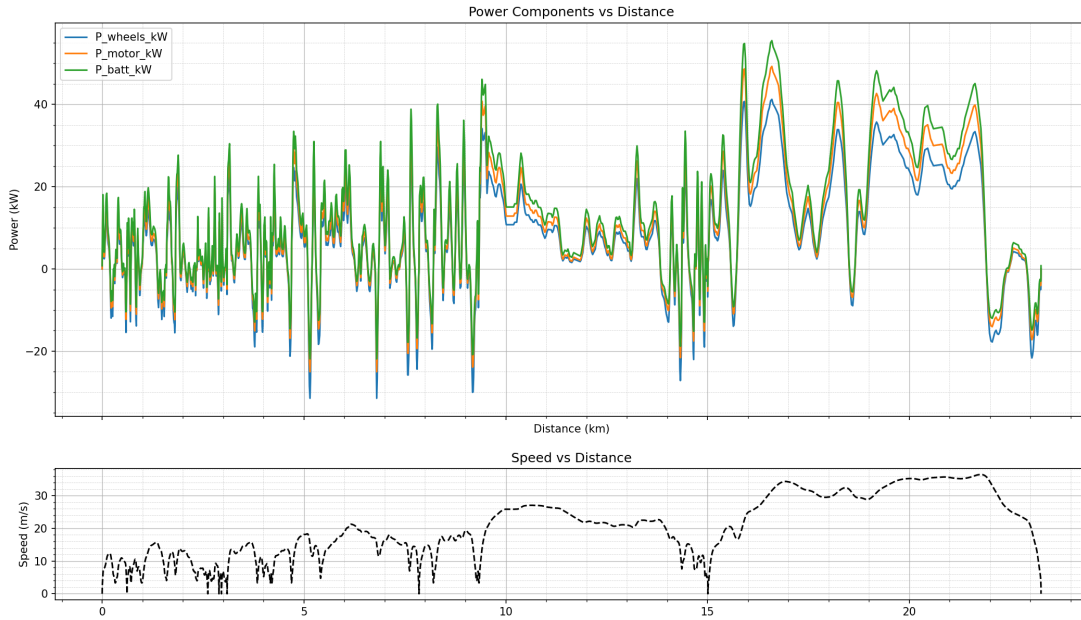


Figure 5.1: Validation of BEV Energy Consumption Model on WLTC cycle

## 5.2 Results

The unit test cases described above were used to evaluate the functionality of each module independently. After validating the modules in isolation, they were integrated and tested at the system level. To thoroughly evaluate system performance, we stress-test the pipeline using different types of routes to ensure that all modules work together to produce realistic velocity, power, and SOC trajectories across diverse driving environments. For this purpose, the following driving route test cases were selected:

- **Urban Route:** Frequent stops, high regeneration events, and lower average speeds leading to increased energy consumption due to frequent braking and acceleration.
- **Freeway Route:** Smooth velocity profiles with minimal braking resulting in lower energy consumption and stable SOC decline.
- **Hilly Terrain Route:** Significant elevation changes and curvy roads with high energy demand in uphill sections, reduced energy consumption in downhill sections due to regenerative braking.

A brief description of the testing process, module flow in the pipeline, and the parameters used for the modules is provided below:

1. A Valhalla routing engine server instance was deployed using Docker for the state of Michigan, using the OSM data file available at <https://download.geofabrik.de/north-america/us/michigan.html>. Michigan was chosen due to the author’s familiarity with local routes, enabling subjective evaluation. Larger map regions can also be deployed, but they require additional computational resources.
2. The test route is selected using the Map Display Handler, which generates a GeoJSON file containing the GPS coordinates of the chosen route.
3. The Route Processor module retrieves the node and edge data for the GPS coordinates from the geoJSON. It is configured to encode 70 coordinates per request to the Valhalla APIs and to decode and process the returned data in parallel.
4. The retrieved edge data is passed as a Pandas DataFrame to the Velocity Profile Generator module, which constructs a distance based reference velocity profile. This reference profile is then provided to the PID Controller module.
5. The PID Controller module simulates vehicle states using the reference velocity profile and forwards the simulated states to the Driver LSTM module. A step size of 10 milliseconds is used for Euler integration within the PID controller.
6. The Driver LSTM module discretizes the route into evenly spaced GPS coordinates at 1-meter intervals and again uses the Route Processor to retrieve node and edge data. The simulated vehicle states from the PID controller are interpolated onto these discretized coordinates. Sequences of past vehicle states, past road context, and future road context are generated using a stride of 10 steps (10 meters) and passed to the LSTM model for batch inference with a batch size of 64. The predicted driver velocity profile is then filtered using a first-order Butterworth low-pass filter followed by zero-phase forward-backward filtering. A filter time constant of 8 m and a sampling interval of 1 m are used. Although the filters are designed for time-domain signals, the filter constants here are defined in meters because the input is a distance-domain signal.

7. The predicted driver-specific velocity profile and elevation data are then passed, in the distance domain, to the BEV Energy Consumption Model, which computes the total energy consumption for the route and the SOC evolution based on the predicted velocity. The model uses the Nissan Leaf parameters defined in [bev'paper] for the calculations which are given below:

- $m = 1595 \text{ kg}$
- $\rho = 1.22563 \text{ kg/m}^3$
- $A_f = 2.3316 \text{ m}^2$
- $C_D = 0.28$
- $C_r = 1.75$
- $C_1 = 0.0328$
- $C_2 = 4.575$
- $g = 9.8066 \text{ m/s}^2$
- $\eta_{\text{driveline}} = 0.92$
- $\eta_{\text{motor}} = 0.91$
- $\eta_{\text{batt}} = 0.90$
- $\eta_{\text{rbmax}} = 0.95$
- $\alpha = 0.0411$
- $P_{\text{aux}} = 700 \text{ W}$
- $C_W = 62000 \text{ Wh}$

The following subsections present the results corresponding to each of the selected driving-route test cases.

### 5.2.1 Urban Route

Urban routes are characterized by frequent stops caused by traffic lights, stop signs, and street intersections. This results in repeated acceleration and deceleration events, leading to higher overall energy consumption. For this test case, we evaluate the model on a short route in the Detroit downtown area, shown in Figure 5.2.

The driver-specific predicted velocity in Figure 5.3 shows that the vehicle does not always come to a complete stop for all conditional events such as traffic signals, yield signs, and stop signs, unlike the simulated velocity labeled as *target/actual speed*. The simulated velocity from the PID Controller module labeled as 'Target/Actual Speed' reaches 0 mph for all such events because it is explicitly programmed to do so. In contrast, the predicted velocity from the LSTM model typically drops below 5 mph but does not reach zero in many cases. This behavior is likely due to the limited

future context (100 m) chosen in the LSTM architecture, which may be insufficient to perfectly model full vehicle stops. Additionally, the ground-truth dataset used for training does not always contain zero-speed samples for traffic signals or yield signs. As a result, the LSTM model appears to have learned that a full stop is not always required—an interpretation that aligns with real-world driving, where green traffic lights or yield signs during low traffic conditions often eliminate the need to stop. Nevertheless, the model correctly predicts deceleration behavior near traffic signals and at intersections with large heading changes (i.e., left or right turns).

The acceleration profile in Figure 5.4 further illustrates the differences between the simulated and predicted profiles. The simulated acceleration from the PID Controller saturates at the predefined maximum and minimum limits, applying maximum acceleration during speed increases and maximum deceleration during braking. The driver-specific predicted acceleration, however, does not exhibit such saturation and instead reflects smoother, more realistic driving behavior. Both the predicted velocity and acceleration contain natural variations, unlike the smooth simulated profiles, capturing the variability present in real-world driving and the patterns learned from the training data.

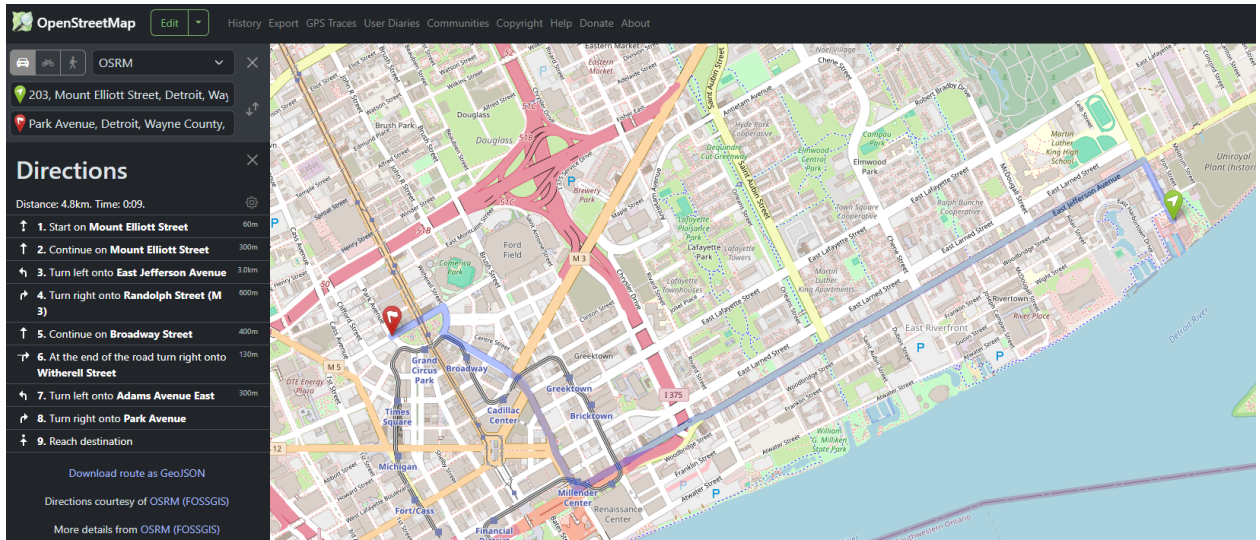


Figure 5.2: Map View of the chosen Urban Route

Figure 5.5 shows the power produced and consumed at the wheels, motor, and battery, while Figure 5.6 shows the corresponding State of Charge (SOC) reduction and energy consumption over distance travelled. Positive acceleration results in positive power demand, whereas negative acceleration produces negative power due to regeneration. The rate of SOC depletion and energy consumption correlates directly with these power fluctuations

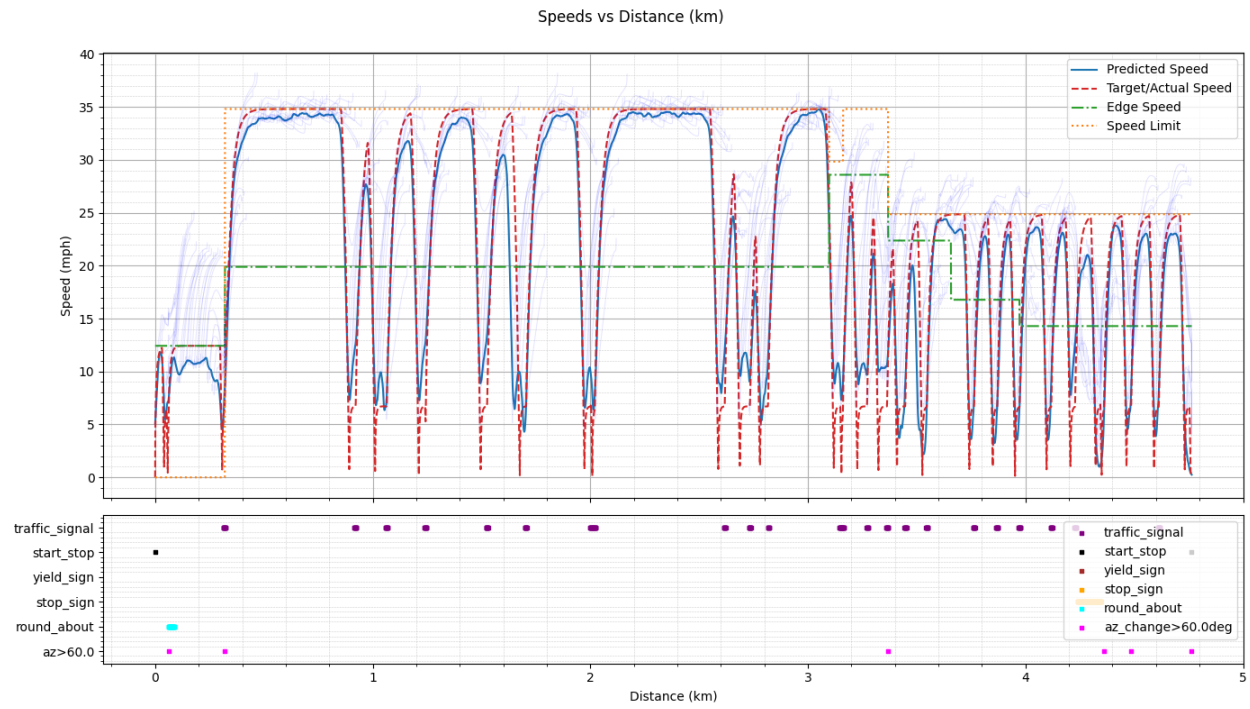


Figure 5.3: Predicted Velocity for the chosen Urban Route

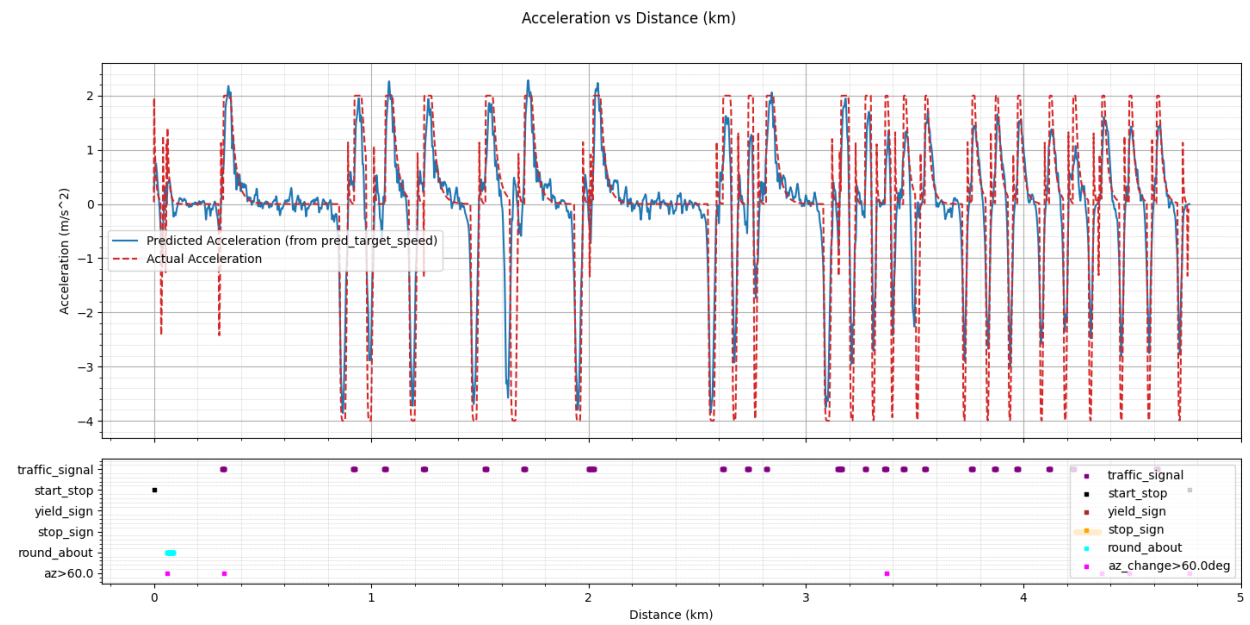


Figure 5.4: Predicted Acceleration for the chosen Urban Route



Figure 5.5: Predicted power demand for the chosen Urban Route

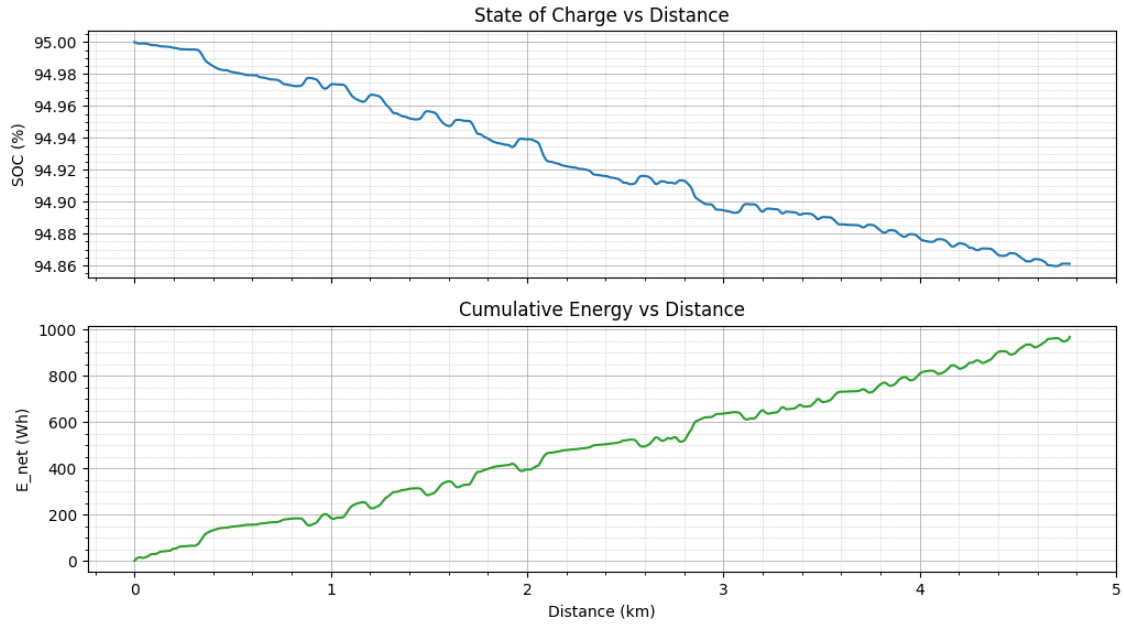


Figure 5.6: Predicted energy Consumption for the chosen Urban Route



## 5.2.2 Freeway Route

Freeway routes typically produce smooth velocity profiles due to consistent speed limits over long stretches. This leads to minimal braking, lower energy consumption, and a stable SOC decline. For this test case, we evaluate the model on a route between Canton, Michigan and Ann Arbor, Michigan, which consists primarily of freeway driving with short urban segments at the beginning and end, as shown in Figure 5.7

The driver-specific predicted velocity in Figure 5.8 oscillates around the legal speed limits, with small variations reflecting natural driver behavior. These fluctuations are likely generated by the LSTM model as part of its learned driving style. In contrast, the simulated velocity profile from the PID Controller (labeled *target/actual speed*) is smooth and does not include such variations. These oscillations are also visible in the predicted acceleration profile in Figure 5.9.

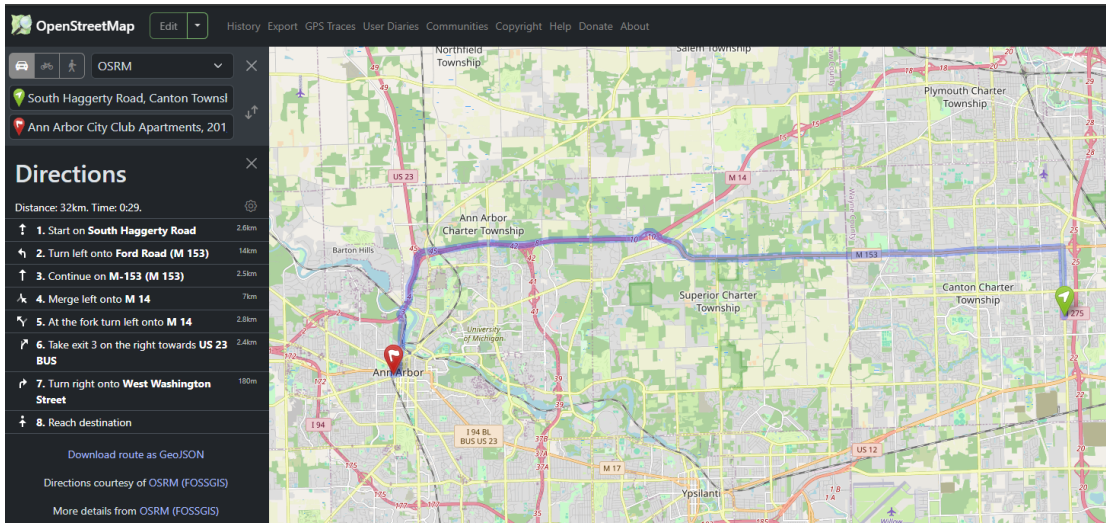


Figure 5.7: Map View of the chosen Freeway Route

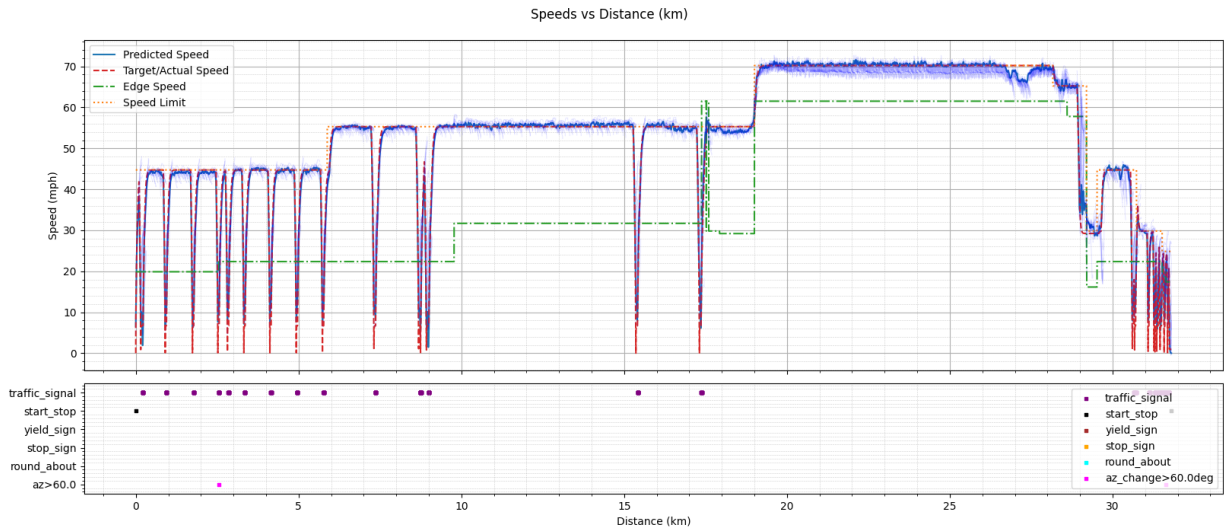


Figure 5.8: Predicted Velocity for the chosen Freeway Route

Figure 5.10 shows the power produced or consumed at the wheels, motor, and battery, while

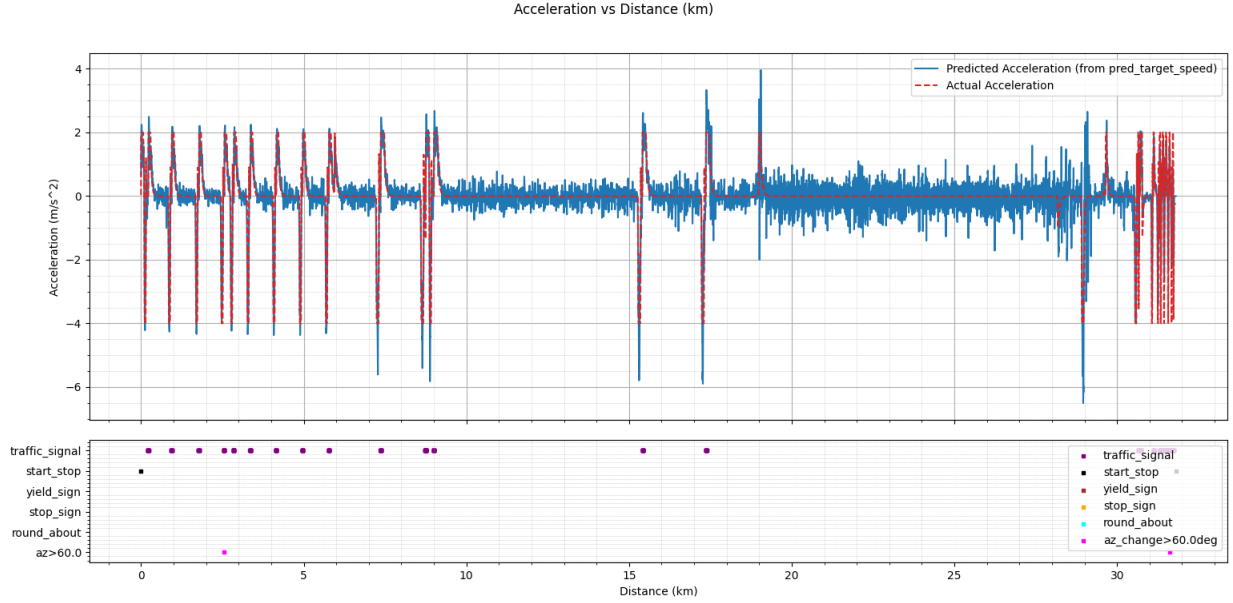


Figure 5.9: Predicted Acceleration for the chosen Freeway Route

Figure 5.11 shows the SOC reduction and energy consumption over distance for the freeway route. As expected, power oscillates uniformly around 0W, and both SOC and energy consumption evolve nearly linearly along the freeway sections.

### 5.2.3 Hilly Terrain Route

Hilly terrain routes are characterized by significant elevation changes and winding roads, with high energy demand during uphill segments and reduced energy consumption during downhill segments due to regenerative braking. For this test case, we evaluate the model on a route between two mountain peaks in the Porcupine Mountains in Michigan’s Upper Peninsula, shown in Figure 5.12. The route begins with a downhill section followed by a sustained uphill climb.

The driver-specific predicted velocity in Figure 5.13 shows the subtle influence of elevation changes on the vehicle speed. The corresponding predicted acceleration profile is shown in Figure 5.14

Figure 5.15 shows the power produced or consumed at the wheels, motor, and battery, while Figure 5.16 shows the SOC reduction and energy consumption over distance traveled for the hilly terrain route. In the initial downhill section, the power is predominantly negative due to continuous regeneration. In contrast, during the uphill section, power remains positive even when acceleration is negative, reflecting the additional energy required to overcome gravity. Consequently, the downhill portion exhibits slow SOC depletion, while the uphill portion shows a much faster rate of energy consumption and SOC decline.



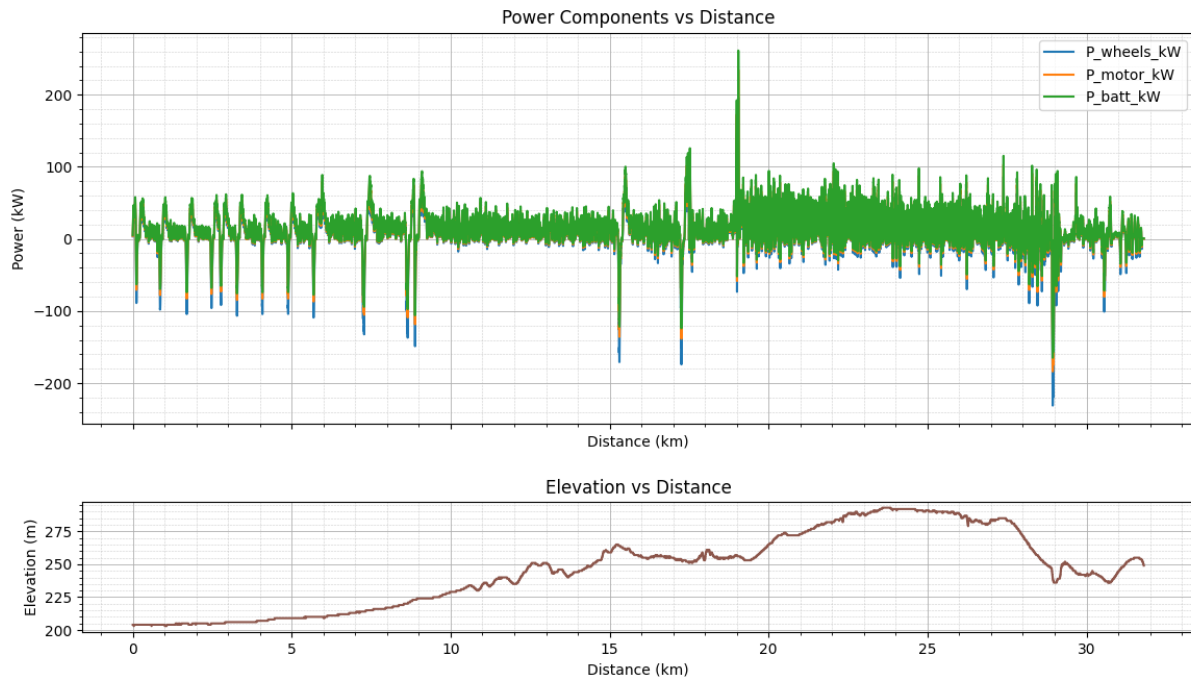


Figure 5.10: Predicted power demand for the chosen Freeway Route

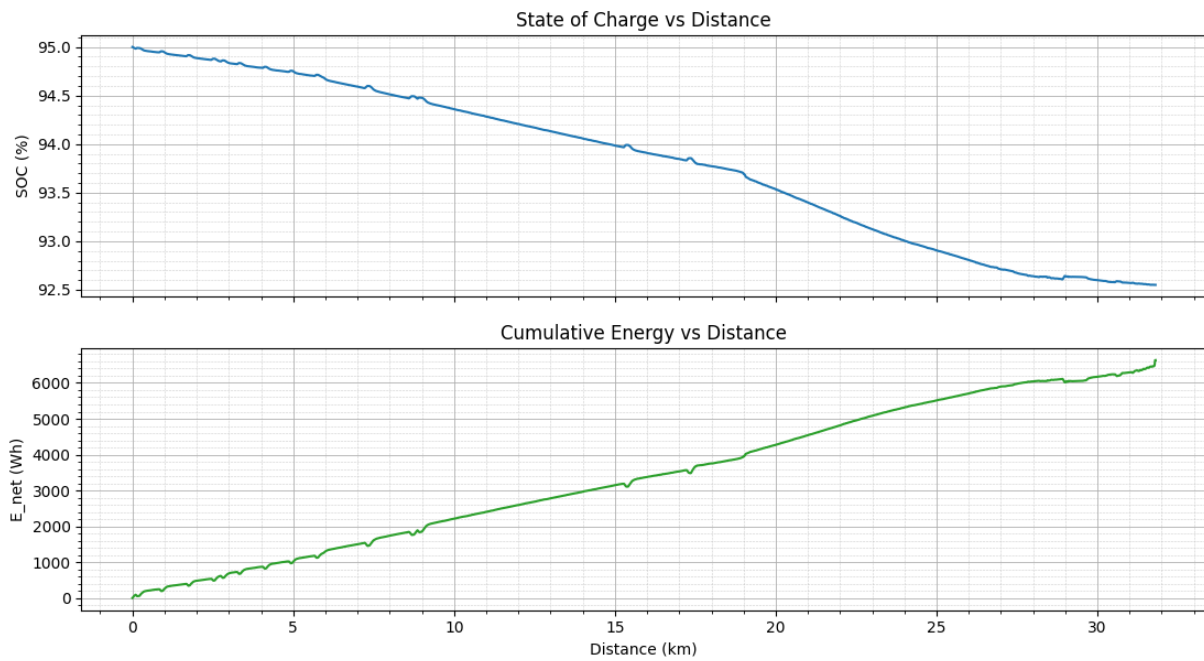


Figure 5.11: Predicted energy Consumption for the chosen Freeway Route

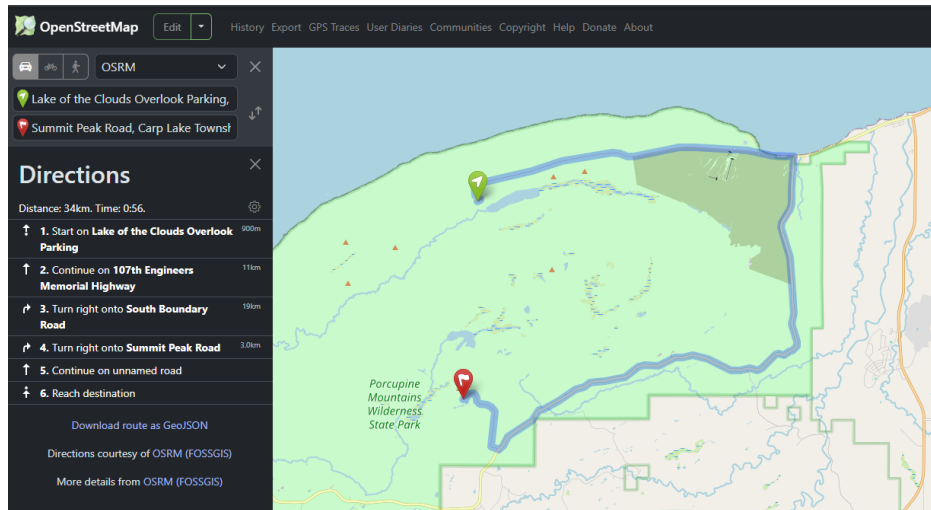


Figure 5.12: Map View of the chosen Hilly Terrain Route

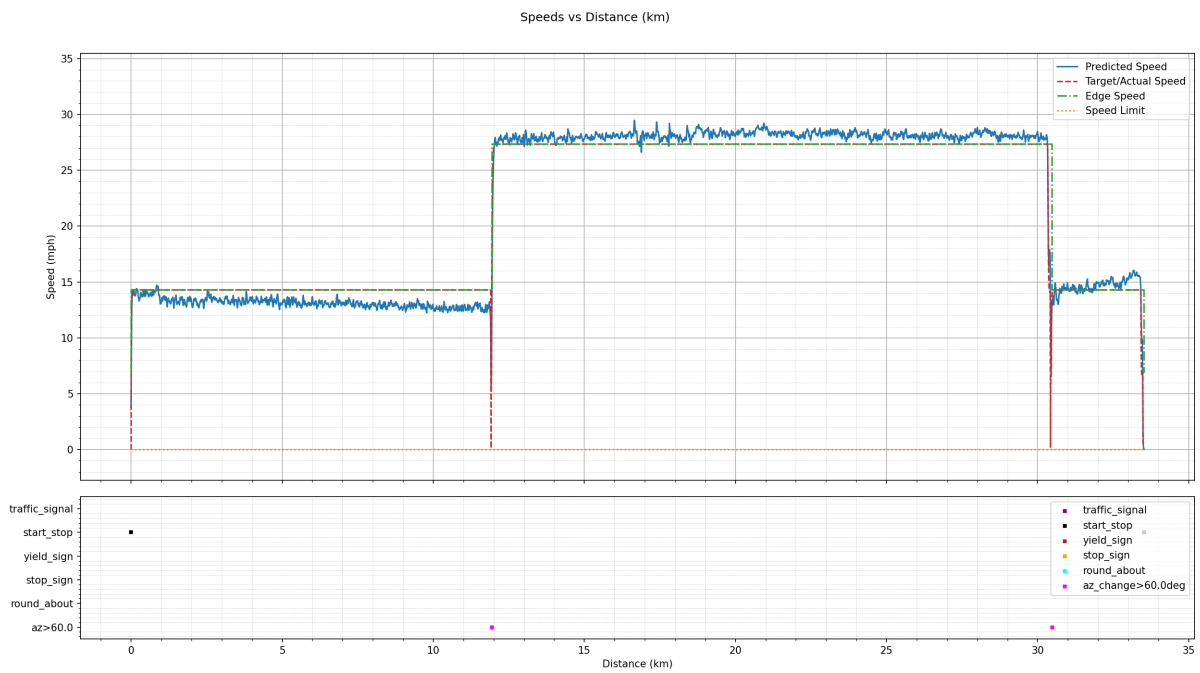


Figure 5.13: Predicted Velocity for the chosen Hilly Terrain Route

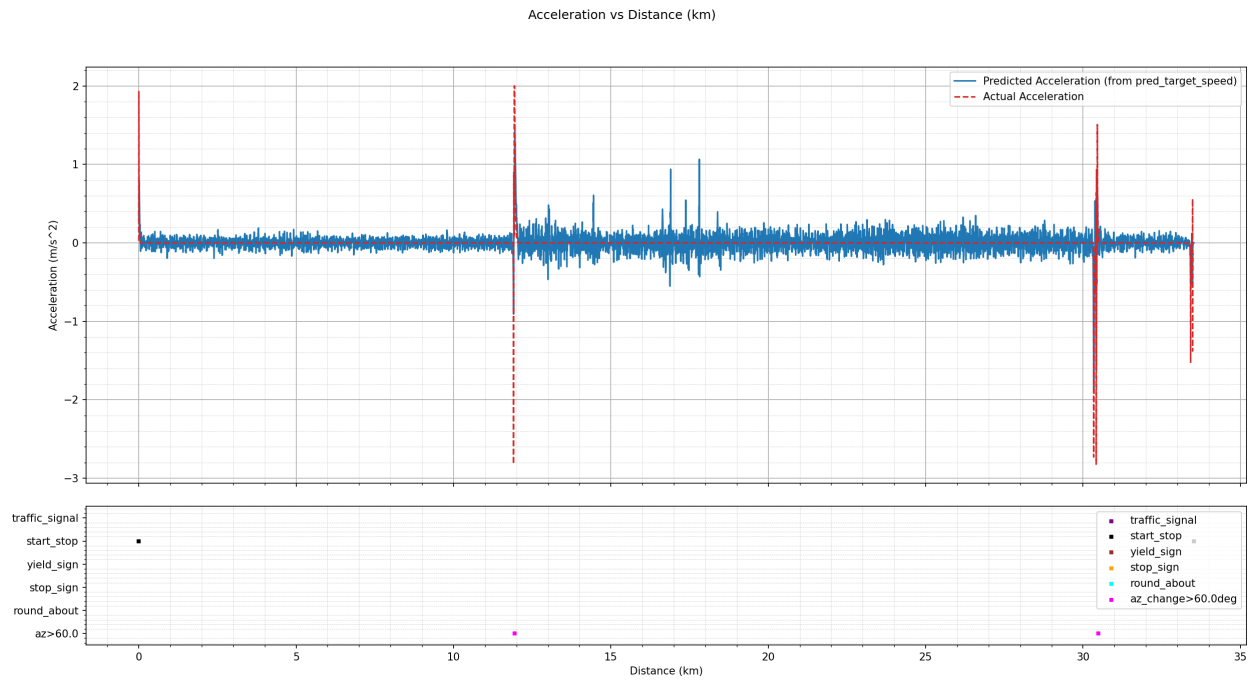


Figure 5.14: Predicted Acceleration for the chosen Hilly Terrain Route

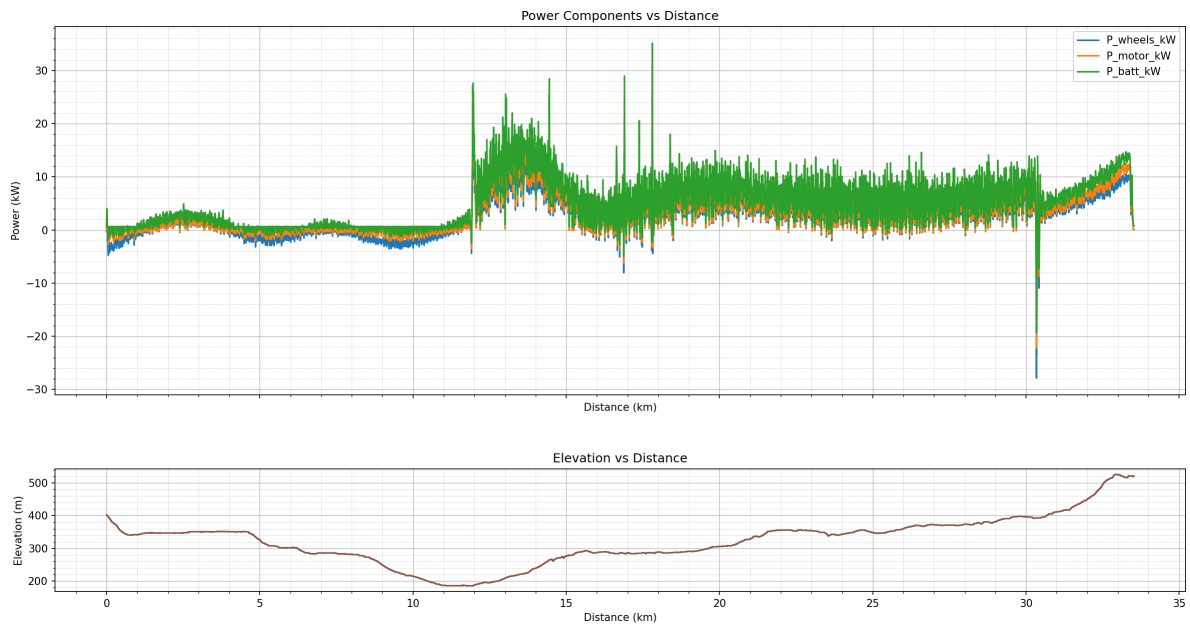


Figure 5.15: Predicted power demand for the chosen Hilly Terrain Route

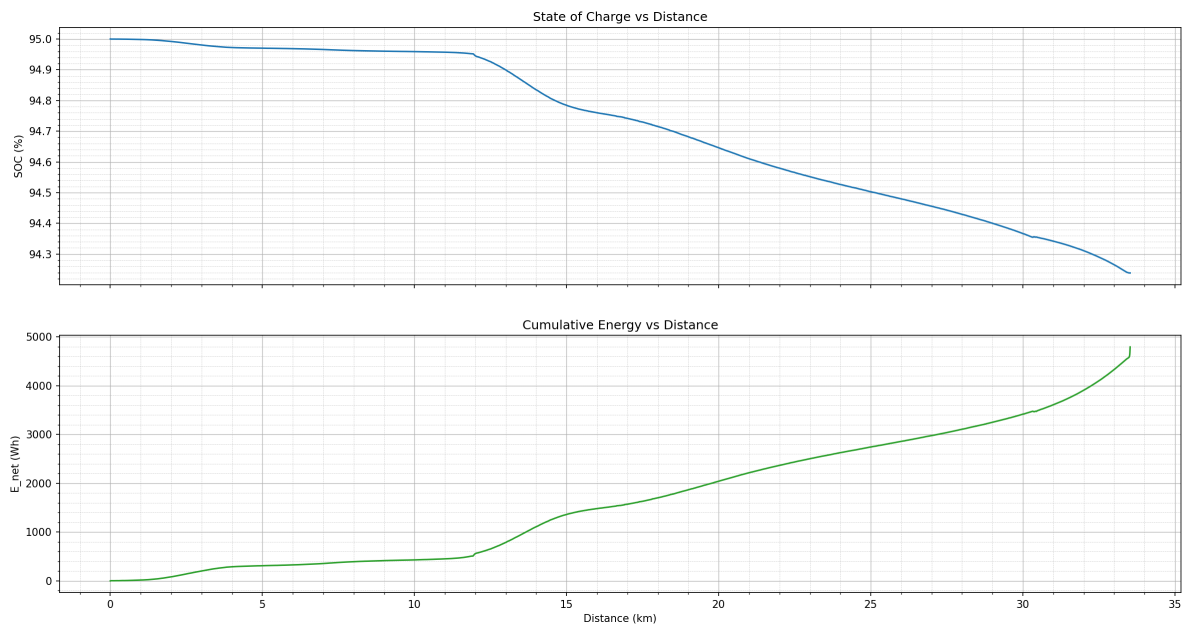


Figure 5.16: Predicted energy Consumption for the chosen Hilly Terrain Route

## Chapter 6

# Conclusions and Future work

This work presented an end-to-end framework for generating driver-specific velocity profiles and estimating the corresponding energy consumption and State of Charge (SOC) evolution for Battery Electric Vehicles (BEVs). The system integrates multiple modules—including the Route Processor, Velocity Profile Generator, PID Controller, Driver LSTM, and the BEV Energy Consumption Model—into a unified pipeline capable of producing realistic, high-resolution predictions across diverse driving environments. Through a combination of unit-level validation and system-level testing on urban, freeway, and hilly terrain routes, the framework demonstrated its ability to capture meaningful aspects of driver behavior, vehicle dynamics, and energy usage.

The results show that the Driver LSTM model successfully learns and reproduces key behavioral patterns present in the training data. In urban environments, the model predicts nuanced deceleration behavior near intersections and turns, even though it does not always bring the vehicle to a complete stop. This behavior reflects both the limitations of the training dataset—which rarely contains true zero-speed events—and the inherent variability of real-world driving. In freeway scenarios, the model produces constant velocity trajectories with natural oscillations around the speed limit, capturing the subtle fluctuations characteristic of human drivers. For hilly terrain routes, the model responds appropriately to elevation changes, with predicted energy consumption and SOC depletion profiles reflecting the influence of gravitational forces. Across all test cases, the BEV Energy Consumption Model produced consistent and interpretable power, energy, and SOC trajectories that aligned with expected physical behavior, such as increased regeneration during downhill segments and higher power demand during uphill climbs.

Overall, the system demonstrates that combining learned driver behavior with physics-based energy modeling can yield realistic and context-aware predictions of BEV performance. The modular design also enables flexible experimentation, scalability, and integration with additional data sources or modeling components.

While the results are promising, several opportunities exist to further improve prediction ac-

curacy and fidelity of the modeled driver behavior. First, the LSTM model’s ability to predict full vehicle stops could be enhanced by using an LSTM architecture that considers longer horizon for the future context features and by augmenting the training dataset with more examples of stop-and-go behavior.

Second, the current architecture depends on a fixed stride and window size. Exploring attention-based models, transformer architectures, or variable-length sequence encoders could enable the system to capture longer-range dependencies and more nuanced driving behaviors. Variable-length sequences are particularly appropriate because the driver’s available future context is not constant. For instance, at higher speeds the effective look-ahead horizon is much larger than at lower speeds. An encoder that adapts its temporal horizon based on driving speed would therefore provide a more efficient and context-aware prediction of future events. Transformer models are especially well suited here, as they excel at modeling long-term dependencies and often outperform LSTMs in such settings.

Third, the energy consumption model can be further enhanced by incorporating detailed efficiency maps for individual drivetrain components, as well as more comprehensive representations of battery management and auxiliary loads.

Fourth, integrating real-time or historical traffic data such as congestion levels, signal timing patterns, and queue lengths—could significantly improve prediction accuracy in urban environments. Traffic conditions, which this framework does not consider, strongly influence stop-and-go behavior, acceleration patterns, and regenerative braking opportunities, and therefore have a direct impact on energy consumption.

Finally, expanding the evaluation to a broader set of routes, drivers, and environmental conditions would strengthen the generalization of the model and highlight additional areas for refinement.

In summary, this work establishes a comprehensive framework for driver-specific velocity prediction and BEV energy modeling, demonstrating strong performance across multiple driving contexts. With targeted enhancements to the model architecture, training data, and contextual inputs, the system can be further improved to deliver even more accurate and realistic predictions, ultimately supporting applications in eco-routing, energy-aware navigation, and personalized range estimation.

# Bibliography

- [1] Roberto Álvarez, Alberto López, and Nieves De la Torre. “Evaluating the effect of a driver’s behaviour on the range of a battery electric vehicle”. In: *Proceedings of the Institution of Mechanical Engineers Part D Journal of Automobile Engineering* 229 (10 Dec. 2014), pp. 1379–1391. ISSN: 0954-4070, 2041-2991. DOI: 10.1177/0954407014561483. URL: <https://doi.org/10.1177/0954407014561483>.
- [2] Chris Bingham, Chris Walsh, and Steve Carroll. “Impact of driving characteristics on electric vehicle energy consumption and range”. In: *IET Intelligent Transport Systems* 6 (1 Feb. 2012), pp. 29–35. ISSN: 1751-956X, 1751-9578. DOI: 10.1049/iet-its.2010.0137. URL: <https://doi.org/10.1049/iet-its.2010.0137>.
- [3] Jan Dedek et al. “BEV Remaining Range Estimation Based on Modern Control Theory - Initial Study”. In: *IFAC-PapersOnLine* 52 (27 Jan. 2019), pp. 86–91. ISSN: 2405-8963, 2405-8971. DOI: 10.1016/j.ifacol.2019.12.738. URL: <https://doi.org/10.1016/j.ifacol.2019.12.738>.
- [4] Felix Deufel, Martin Gießler, and Frank Gauterin. “A generic prediction approach for optimal control of electrified vehicles using artificial intelligence”. In: *Vehicles* 4.1 (2022), pp. 182–198.
- [5] Felix Deufel et al. “Velocity prediction based on map data for optimal control of electrified vehicles using recurrent neural networks (lstm)”. In: *Vehicles* 4.3 (2022), pp. 808–824.
- [6] SciPy Developers. *scipy.signal.butter* — Butterworth digital and analog filter design. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.butter.html>. Accessed: 2026-02-16.
- [7] SciPy Developers. *scipy.signal.filtfilt* — Zero-phase forward and reverse digital filtering. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.filtfilt.html>. Accessed: 2026-02-16.
- [8] Selenium Developers. *Selenium WebDriver for Python*. Accessed: 12 June 2025. 2025. URL: [https://www.selenium.dev/documentation/webdriver/getting\\_started/](https://www.selenium.dev/documentation/webdriver/getting_started/).
- [9] Valhalla Developers. *Valhalla - Open Source Routing Engine*. Accessed: 12 June 2025. 2025. URL: <https://github.com/valhalla/valhalla>.
- [10] Valhalla Developers. *Valhalla Routing Engine Documentation*. Accessed: 12 June 2025. 2025. URL: <https://valhalla.github.io/valhalla/>.

- [11] Chiara Fiori, Kyoungcho Ahn, and Hesham A Rakha. “Power-based electric vehicle energy consumption model: Model development and validation”. In: *Applied Energy* 168 (2016), pp. 257–268.
- [12] Google. *ChromeDriver - WebDriver for Chrome*. Accessed: 12 June 2025. 2025. URL: <https://sites.google.com/chromium.org/driver/>.
- [13] Jianhua Guo et al. “A novel energy consumption prediction model with combination of road information and driving style of BEVs”. In: *Sustainable Energy Technologies and Assessments* 42 (2020), p. 100826.
- [14] Paul Heckelmann, Sandro Chris Breuer, and Stephan Rinderknecht. “Investigation of different LSTM-based encoder-decoder neural networks for vehicle speed prediction”. In: *Scientific Reports* 15.1 (2025), p. 32662.
- [15] Jianfeng Hou et al. “Vehicle Trajectory Data Augmentation Using Data Features and Road Map”. In: *Electronics* 14.14 (2025), p. 2755.
- [16] Weicheng Hou et al. “Multiobjective Optimization of Large-Scale EVs Charging Path Planning and Charging Pricing Strategy for Charging Station”. In: *Complexity* 2021 (Jan. 2021), pp. 1–17. ISSN: 1076-2787, 1099-0526. DOI: 10.1155/2021/8868617. URL: <https://doi.org/10.1155/2021/8868617>.
- [17] Md Ahasan Kabir. “Vehicle speed prediction based on road status using machine learning”. In: *Advanced Research in Energy and Engineering* 2.1 (2020).
- [18] Zhenhong Lin. “Measuring Range Anxiety: the Substitution-Emergency-Detour (SED) Method”. In: *World Electric Vehicle Journal* 5 (1 Mar. 2012), pp. 8–13. ISSN: 2032-6653. DOI: 10.3390/wevj5010008. URL: <https://doi.org/10.3390/wevj5010008>.
- [19] Lan Liu, Liling Zhu, and Da Yang. “Modeling and simulation of the car-truck heterogeneous traffic flow based on a nonlinear car-following model”. In: *Applied Mathematics and Computation* 273 (2016), pp. 706–717.
- [20] Chen Lv et al. “Driving-style-based codesign optimization of an automated electric vehicle: A cyber-physical system approach”. In: *IEEE Transactions on Industrial Electronics* 66.4 (2018), pp. 2965–2975.
- [21] Jeremy Neubauer and Eric Wood. “The impact of range anxiety and home, workplace, and public charging infrastructure on simulated battery electric vehicle lifetime utility”. In: *Journal of Power Sources* 257 (Jan. 2014), pp. 12–20. ISSN: 0378-7753, 1873-2755. DOI: 10.1016/j.jpowsour.2014.01.075. URL: <https://doi.org/10.1016/j.jpowsour.2014.01.075>.
- [22] OpenStreetMap contributors. *OpenStreetMap - Collaborative Mapping Project*. Accessed: 12 June 2025. 2025. URL: <https://www.openstreetmap.org>.
- [23] Walaa Othman et al. “DriverMVT: In-Cabin Dataset for Driver Monitoring including Video and Vehicle Telemetry Information”. In: *Data* 7 (5 May 2022), pp. 62–62. ISSN: 2306-5729. DOI: 10.3390/data7050062. URL: <https://doi.org/10.3390/data7050062>.



- [24] Luca Paparusso, Stefano Melzi, and Francesco Braghin. “Real-time forecasting of driver-vehicle dynamics on 3D roads: A deep-learning framework leveraging Bayesian optimisation”. In: *Transportation research part C: emerging technologies* 156 (2023), p. 104329.
- [25] Ahmet Alperen Polat et al. “Analysis of Factors Affecting Electric Vehicle Range Estimation: A Case Study of the Eskisehir Osmangazi University Campus”. In: *Sustainability* 17 (8 Apr. 2025), pp. 3488–3488. ISSN: 2071-1050. DOI: 10.3390/su17083488. URL: <https://doi.org/10.3390/su17083488>.
- [26] Heran Shen et al. “Personalized velocity and energy prediction for electric vehicles with road features in consideration”. In: *IEEE Transactions on Transportation Electrification* 9.3 (2023), pp. 3958–3969.
- [27] Vincent R. Tannahill, Kashem M. Muttaqi, and Danny Sutanto. “Driver alerting system using range estimation of electric vehicles in real time under dynamically varying environmental conditions”. In: *IET Electrical Systems in Transportation* 6 (2 July 2015), pp. 107–116. ISSN: 2042-9738, 2042-9746. DOI: 10.1049/iet-est.2014.0067. URL: <https://doi.org/10.1049/iet-est.2014.0067>.
- [28] Chalermchat Theeraviriya, Worapot Sirirak, and Natthanan Praseeratasang. “Location and Routing Planning Considering Electric Vehicles with Restricted Distance in Agriculture”. In: *World Electric Vehicle Journal* 11 (4 Sept. 2020), pp. 61–61. ISSN: 2032-6653. DOI: 10.3390/wevj11040061. URL: <https://doi.org/10.3390/wevj11040061>.
- [29] Juan D. Valladolid et al. “A Novel Energy-Efficiency Optimization Approach Based on Driving Patterns Styles and Experimental Tests for Electric Vehicles”. In: *Electronics* 10 (10 May 2021), pp. 1199–1199. ISSN: 2079-9292. DOI: 10.3390/electronics10101199. URL: <https://doi.org/10.3390/electronics10101199>.
- [30] Jiri Vraný, Michal Krepelka, and Matej Chumlen. “Generating synthetic vehicle speed records using lstm”. In: *IFIP International Conference on Artificial Intelligence Applications and Innovations*. Springer. 2023, pp. 125–136.
- [31] Jiri Vraný et al. *Vehicle speed dataset*. 2023. DOI: 10.21227/n1z9-e630. URL: <https://dx.doi.org/10.21227/n1z9-e630>.
- [32] Jiquan Wang, Igo Besselink, and Henk Nijmeijer. “Electric vehicle energy consumption modelling and prediction based on road information”. In: *World Electric Vehicle Journal* 7.3 (2015), pp. 447–458.
- [33] Rui Wang and Srdjan M Lukic. “Review of driving conditions prediction and driving style recognition based control algorithms for hybrid electric vehicles”. In: *2011 IEEE Vehicle power and propulsion conference*. IEEE. 2011, pp. 1–7.
- [34] Yazan Al-Wreikat, Clara Serrano, and José Ricardo Sodr . “Driving behaviour and trip condition effects on the energy consumption of an electric vehicle under real-world driving”. In: *Applied Energy* 297 (May 2021), pp. 117096–117096. ISSN: 0306-2619, 1872-9118. DOI: 10.1016/j.apenergy.2021.117096. URL: <https://doi.org/10.1016/j.apenergy.2021.117096>.

- [35] Fei Wu et al. “Study on Factors Affecting the Range of Electric Vehicle with ADVISOR”. In: *Advanced materials research* (Dec. 2013), pp. 1092–1095. ISSN: 1022-6680, 1662-8985. DOI: 10.4028/www.scientific.net/amr.860-863.1092. URL: <https://doi.org/10.4028/www.scientific.net/amr.860-863.1092>.
- [36] Xinkai Wu et al. “Electric vehicles’ energy consumption measurement and estimation”. In: *Transportation Research Part D: Transport and Environment* 34 (2015), pp. 52–67.
- [37] Hao Ye et al. “Machine Learning for Vehicular Networks”. In: *arXiv (Cornell University)* (Jan. 2017). DOI: 10.48550/arxiv.1712.07143. URL: <https://arxiv.org/abs/1712.07143>.
- [38] Li Yufang, Chen Mingnuo, and Zhao Wanzhong. “Investigating long-term vehicle speed prediction based on BP-LSTM algorithms”. In: *IET Intelligent Transport Systems* 13.8 (2019), pp. 1281–1290.
- [39] Chunjie Zhai, Fei Luo, and Yonggui Liu. “A novel predictive energy management strategy for electric vehicles based on velocity prediction”. In: *IEEE Transactions on Vehicular Technology* 69.11 (2020), pp. 12559–12569.
- [40] Laura Zheng et al. “Quantifying and Modeling Driving Styles in Trajectory Forecasting”. In: (Jan. 2025). DOI: 10.48550/ARXIV.2503.04994. URL: <https://arxiv.org/abs/2503.04994>.